

Задачи с собеседований по Objective-C

Junior · Middle · Senior — с полным разбором решений

Учебник «Objective-C с нуля»

Содержание

Приложение С. Как готовиться к собеседованию по Objective-C	1
Где сегодня собеседуют на Objective-C	1
Что проверяют по уровням	2
Junior — раздел D	2
Middle — раздел E	2
Senior — раздел F	3
Типичные форматы вопросов	3
Классические устные ловушки Objective-C	4
Как готовиться	6
Как разбирать задачи в этом разделе	7
Напоследок	8
Документация Apple	8
Приложение D. Задачи собеседования: Junior	10
Задача 1. Что выведет [nil length]	10
Задача 2. == или isEqualToString:	11
Задача 3. Два NSNumber и ==	12
Задача 4. Что делают alloc и init и что будет, если забыть init	13
Задача 5. strong против copy у свойства NSString	14
Задача 6. BOOL, YES, NO и ловушка с числом	15
Задача 7. addObject:nil — что произойдёт	16
Задача 8. objectAtIndex: против firstObject на пустом массиве	17
Задача 9. nil-значение в словаре и NSNull	17
Задача 10. Почему нельзя [array addObject:5]	18
Задача 11. Быстрый перебор for-in и мутация во время него	19
Задача 12. Полиморфизм: какой метод вызовется	20
Задача 13. isKindOfClass: против isKindOfClass:	21
Задача 14.instancetype или id в фабричном методе	22
Задача 15. Реализуй стек поверх NSMutableArray	23
Задача 16. Найди утечку: сильная ссылка на delegate	25
Задача 17. mutableCopy и copy — что можно менять	26
Задача 18. Объект-значение как ключ словаря	27
Задача 19. Свойство readonly и приватная изменяемость	27
Задача 20. Что такое self в методе и зачем [super init]	28
Что повторить перед собеседованием	29
Документация Apple	30
Приложение E. Задачи собеседования: Middle	31

Задача 1. Что выведет блок: захват по значению и <code>__block</code>	31
Задача 2. <code>weak</code> после смерти объекта и ловушка <code>unsafe_unretained</code>	33
Задача 3. Найди утечку: команда и игроки	34
Задача 4. Цикл удержания объект ☒ блок: что выведет <code>dealloc</code>	35
Задача 5. Зачем <code>@autoreleasepool</code> внутри длинного цикла	36
Задача 6. Поверхностная копия коллекции	37
Задача 7. <code>copy</code> -семантика у <code>NSString</code> -свойства	38
Задача 8. Простой кэш на <code>NSMutableDictionary</code> и <code>copy</code> -ключи	39
Задача 9. Спроектируй делегат с необязательными методами	40
Задача 10. Две категории с одинаковым методом — что вызовется	41
Задача 11. Категория не может добавить <code>ivar</code> — как обойти	42
Задача 12. KVC: доступ по строке, <code>keyPath</code> и операторы	43
Задача 13. KVO: что приходит в <code>observeValueForKeyPath:</code> и почему <code>ivar</code> молчит	44
Задача 14. Реализуй <code>NSCopying</code>	45
Задача 15. Реализуй <code>isEqual:</code> и <code>hash</code> вместе	46
Задача 16. <code>isEqual:</code> против <code>isEqualToString:</code>	47
Задача 17. Быстрый разбор: сработает ли <code>dealloc</code> ?	48
Задача 18. Code review: найди и почини утечку в блоке	48
Что проверяет этот раздел	50
Документация Apple	50
Приложение F. Задачи собеседования: Senior	51
Задача 1. Куда указывает <code>isa</code> класса и где живут методы класса	51
Задача 2. <code>objc_msgSend</code> , <code>SEL</code> , <code>IMP</code> , <code>Method</code> — кто есть кто	53
Задача 3. Динамическое разрешение метода: сколько раз позовётся <code>resolve</code>	54
Задача 4. Логирующий прокси на <code>forwardingTargetForSelector:</code>	55
Задача 5. Рассылающий прокси: почему тут нужен <code>forwardInvocation:</code>	56
Задача 6. Ленивый прокси на <code>NSProxy</code> : зачем отдельный корень	57
Задача 7. <code>Method swizzling</code> правильно: <code>+load</code> , <code>dispatch_once</code> , <code>class_addMethod</code>	58
Задача 8. Найди тонкий баг: <code>swizzling</code> , который ломает KVO	59
Задача 9. Связанный объект: хранимое свойство в категории	60
Задача 10. KVO под капотом: подмена <code>isa</code> и <code>NSKVONotifying_</code>	61
Задача 11. Две категории с одинаковым методом — какая победит	62
Задача 12. Гонка данных: <code>atomic</code> не делает коллекцию потокобезопасной	64
Задача 13. <code>dispatch_sync</code> на текущей последовательной очереди — дедлок	64
Задача 14. Потокобезопасный синглтон на <code>dispatch_once</code>	66
Задача 15. Потокобезопасный кэш: барьерная запись на <code>concurrent</code> -очереди	66
Задача 16. <code>weak</code> в многопоточности и пулы автоосвобождения в фоне	67
Задача 17. Исключение через границу <code>@autoreleasepool</code> и в фоновом потоке	69
Задача 18. <code>Tagged pointers</code> : почему <code>==</code> для маленьких <code>NSNumber</code> иногда прав	71
Задача 19. Найди и почини утечку: цикл удержания через <code>NSTimer</code>	72
Что повторить перед Senior-собеседованием	73
Документация Apple	74

Приложение С. Как готовиться к собеседованию по Objective-C

Книга кончилась, а вопросы на собеседовании только начинаются. Это приложение — мостик от «я понимаю язык» к «я могу про него внятно рассказать вслух, под взглядом человека напротив». Это два разных умения. Можно годами писать рабочий код и поплыть на простом «а почему свойство NSString объявляют через `copy?`». Здесь мы это закроем.

Приложение прозаическое — кода почти нет. Сам код, который надо писать руками, лежит в следующих трёх разделах (D, E, F) и в папке `code/`. А тут — карта местности: где сегодня вообще собеседуют на Objective-C, что спрашивают на каждом уровне, в каких формах задают вопросы, какие устные ловушки повторяются из раза в раз и как ко всему этому подготовиться так, чтобы не зубрить, а понимать.

Где сегодня собеседуют на Objective-C

Скажу честно, без розовых очков — ровно как говорил во введении и в последней главе. Новый проект с чистого листа в 2020-х обычно начинают на Swift. Если ты ждёшь вакансию «чистый Objective-C, зелёное поле, пишем с нуля» — таких почти не осталось.

Зато живёт и нанимает другое:

- **Поддержка и развитие существующих iOS/macOS-проектов.** Миллионы строк рабочего кода написаны на Objective-C, и их не переписывают одним движением. Их читают, чинят, дополняют. Тут язык нужен не «для общего развития», а каждый день.
- **Смешанные команды Swift + Objective-C.** Большие кодовые базы годами живут на двух языках сразу: новое — на Swift, старое — на Objective-C, и они разговаривают через мост. От тебя ждут, что ты свободен на обоих берегах.
- **Платформенная и инфраструктурная работа.** Библиотеки, SDK, инструменты, где важен runtime: аналитика, логирование, отладочные обвязки. Там Objective-C-механика всплывает даже в Swift-проектах.

Из этого следует важная мысль, которую держи в голове всё собеседование. **Чаще всего вакансии звучит как «iOS-разработчик», а не «Objective-C- разработчик».** Язык спрашивают не сам по себе, а как **фундамент платформы**. Интервьюер хочет понять, понимаешь ли ты, как Apple устроена изнутри: подсчёт ссылок, сообщения, runtime, Foundation. Objective-C здесь — самый честный способ это проверить, потому что в нём вся механика видна руками, без сахара Swift сверху.

Поэтому не удивляйся, если на собеседовании «по Objective-C» половина вопросов будет про вещи, общие со Swift: ARC, циклы удержания, делегаты, многопоточность. Это не подвох. Это и есть фундамент.

Что проверяют по уровням

Дальше идут три раздела с задачами — D, E и F. Они разбиты по уровням: Junior, Middle, Senior. Деление условное (на реальном собеседовании вопросы перемешаны), но оно показывает, **как растёт глубина**. Вот карта, что обычно ждут на каждом уровне.

Junior — раздел D

От джуна хотят уверенного синтаксиса и базовой картины памяти. Не глубины, а отсутствия дыр в основах.

- **Синтаксис сообщений.** [receiver message:arg] — что это, чем отличается от вызова функции, что такое селектор, что такое получатель. Уметь прочесть вложенное сообщение [[obj foo] bar] и не запутаться в скобках.
- **Классы, свойства, инициализация.** @interface/@implementation, методы с - и +, @property, пара alloc/init, что такое назначенный инициализатор (designated initializer), зачем self = [super init].
- **Базовая память.** Что такое ARC и от чего он избавил. Разница strong и weak на пальцах. Почему объектные типы всегда со звёздочкой.
- **Коллекции.** NSArray, NSDictionary, NSSet и их изменяемые варианты. Литералы @[], @{}, @(42). Чем immutable отличается от mutable и что будет, если мутировать immutable.
- **Поведение nil.** Можно ли слать сообщение nil, что вернётся, почему это не падение.

Middle — раздел E

С миддла начинается настоящий разговор про память и про то, как язык устроен чуть глубже поверхности.

- **Память глубже.** Цикл удержания (retain cycle) — как возникает, как его увидеть, как разорвать через weak. Что такое autorelease pool и когда он сливается. Аналог unowned из Swift — `__unsafe_unretained` и чем он опаснее weak.
- **Блоки и захват.** Как блок захватывает переменные, почему self внутри блока — это сильная ссылка и как из-за неё рождается цикл, что делает `__block`, зачем пишут `__weak typeof(self) weakSelf = self`.
- **Протоколы и делегаты.** Паттерн делегирования, почему делегат держат через weak, @required против @optional, проверка respondsToSelector: перед вызовом опционального метода.
- **Категории.** Что можно и чего нельзя добавить категорией, чем она отличается от наследования, что такое расширение класса (class extension) и чем оно отличается от категории.
- **Сору-семантика.** Зачем сору у свойств, разница сору и mutableCopy, поверхностная и глубокая копия, протоколы NSCopying / NSMutableCopying.
- **KVC/KVO базово.** Что такое доступ к свойствам по строковому ключу (valueForKey:), идея наблюдения за изменениями (addObserver:).

Senior — раздел F

Сеньора проверяют на понимание runtime и на умение проектировать, а не только писать.

- **Runtime.** Указатель isa, классы и метаклассы, дерево классов до NSObject, как objc_msgSend находит метод, динамическое разрешение методов, пересылка сообщений (forwarding), method swizzling. Не «слышал слова», а «могу объяснить шаги и нарисовать».
- **Потокобезопасность и GCD.** Очереди (serial/concurrent), dispatch_async, барьеры, гонки данных, почему atomic не спасает коллекцию, как защитить общий ресурс.
- **Проектирование API.** Именование методов в стиле Cocoa, immutable- против mutable-интерфейсов, что выносить в публичный заголовок, а что прятать в class extension.
- **Дизайн-паттерны Cocoa.** Делегирование, target-action, наблюдатель (KVO/NotificationCenter), синглтон через dispatch_once, MVC так, как его понимает Apple.
- **Interop со Swift.** Как класс Objective-C виден в Swift и наоборот, что такое NS_ASSUME_NONNULL, аннотации nullable/nonnull, «облегчённые дженерики» (NSArray<NSString *> *) и почему в runtime от них ничего не остаётся.
- **Краевые случаи.** Поведение nil в арифметике, tagged pointer, toll-free bridging, что возвращает сообщение nil для структуры.

Типичные форматы вопросов

Полезно заранее узнать формы, в которых вопрос прилетает. Их немного.

«**Что выведет этот код**». Тебе показывают листинг и просят сказать результат, не запуская. Проверяют, держишь ли ты в голове модель выполнения: порядок вызовов, поведение `nil`, когда срабатывает геттер, что напечатает `%@` для конкретного объекта. Читай такой код медленно, строка за строкой, проговаривай вслух, что происходит.

«**Найди утечку / найди баг**». Дают кусок кода с retain cycle, с сообщением освобождённому объекту, с мутацией immutable-коллекции, с гонкой данных. Твоя задача — увидеть проблему и объяснить, **почему** она проблема и **как** чинить. Первое, что ищи глазами, — сильные ссылки по кругу (особенно `self` внутри блока, который хранит сам объект).

«**Объясни разницу X и Y**». Любимый жанр. Короткий устный вопрос на понимание. Классические пары:

- `strong` против `copy` для свойства;
- `isEqual:` против `==`;
- `isKindOfClass:` против `isKindOfClass:`;
- `id` против `instancetype`;
- категория против наследования;
- `copy` против `mutableCopy`;
- `nil` против `Nil` против `NULL` против `NSNull`.

Хороший ответ — это **одно точное предложение про суть плюс пример, когда выбор важен**. Не лей воду.

«**Реализуй с нуля**». Просят написать `isEqual:` и `hash` для своего класса, простой синглтон, потокобезопасный кэш, разобрать строку. Здесь важен не только результат, но и то, как ты рассуждаешь по дороге. Проговаривай ход мысли.

Устные концептуальные. Без кода вообще: «расскажи, как работает ARC», «что происходит при отправке сообщения», «что такое autorelease pool». Тут проверяют, умеешь ли ты объяснять — структурно, от общего к частному, своими словами, а не цитатой из документации.

Классические устные ловушки Objective-C

Это вопросы, которые задают снова и снова. Если ты ответишь на них чётко и по-человечески, половина собеседования уже за тобой. Привожу с короткими правильными ответами — не заучивай дословно, а пойми и перескажи своими словами.

Почему свойство `NSString` объявляют через `copy`, а не `strong`? Потому что вместо `NSString` тебе могут передать `NSMutableString` — это его наследник, он подойдёт по типу. Если хранить его через `strong`, ты держишь **тот же самый** изменяемый объект, и внешний код может поменять его у тебя за спиной — твоё «неизменяемое» имя вдруг стало другим. `copy` снимает

неизменяемый снимок в момент присваивания, и он уже никуда не уплывёт. Для изменяемых строк сору вернёт неизменяемую копию — это и есть желаемое поведение свойства.

Чем категория отличается от наследования (subclass)? Категория добавляет методы к **уже существующему** классу, и они появляются сразу у **всех** его экземпляров во всём приложении — даже у тех, что создаёт система. Своих хранимых переменных (ivar) категория добавить не может. Наследование создаёт **новый** тип: ему можно дать новые поля, и изменения видны только у экземпляров подкласса. Коротко: категория — «дописать поведение всем», подкласс — «сделать новую разновидность».

Что такое tagged pointer? Оптимизация runtime на 64-битных системах. Маленькие значения — короткие строки, небольшие числа, даты — система не кладёт в кучу отдельным объектом, а упаковывает **прямо внутрь самого указателя**. То есть указатель перестаёт быть адресом и сам несёт данные, а биты-метки говорят runtime, какого это класса. Выгода — нет аллокации и освобождения, доступ мгновенный. Для тебя как программиста это прозрачно: объект ведёт себя как обычный, просто под капотом его «нет» в куче. Поэтому, кстати, на tagged-pointer-объекты бессмысленно смотреть как на адрес и опасно завязываться на их retainCount.

Можно ли слать сообщение nil? Да, и это не падение — фундаментальное свойство языка. Сообщение nil ничего не делает и возвращает «нулевое» значение: для объекта — nil, для чисел — 0, для BOOL — NO, для структуры — обнулённую структуру. Это и удобно (меньше проверок на nil), и коварно: баг с молчаливым nil не упадёт, а тихо вернёт ноль, и ищи потом, почему «ничего не происходит».

Отличие от Си. В Си вызов через нулевой указатель функции — это мгновенный крах. В Objective-C отправка сообщения nil — штатная безопасная операция, которая просто возвращает ноль. Механику диспетчера сообщений мы разбирали в главе про runtime.

Зачем делегат держат через weak? Чтобы не было цикла удержания. Обычно объект А создаёт объект В и держит его сильно, а В хочет сообщать А о событиях через свойство delegate. Если delegate сильный, А держит В, В держит А — оба никогда не освободятся, утечка. weak разрывает круг: делегат указывает на владельца, но не удерживает его, и автоматически становится nil, когда владелец умирает.

Как работает autorelease pool? Это отложенное освобождение. Когда объект помечен autorelease, он не освобождается сразу — его регистрируют в текущем пуле, а пул сольёт (освободит) всех своих в конце блока @autoreleasepool { ... }. В приложении с UI такой пул автоматически оборачивает каждую итерацию главного цикла событий (run loop), поэтому временные объекты живут до конца обработки события и потом исчезают пачкой. Свой @autoreleasepool руками заводят в тяжёлых циклах, чтобы пиковая память не росла: сливаешь пул на каждой итерации — временные объекты не копятся.

Почему `atomic`-свойства не дают потокобезопасности коллекции? Потому что `atomic` гарантирует атомарность только **одного** обращения — геттер вернёт целый, не разорванный пополам указатель, а сеттер запишет его целиком. И всё. `atomic` ничего не знает про **содержимое** объекта и про **составные** операции. Если два потока одновременно зовут `[array addObject:]` у одного `NSMutableArray`, само свойство атомарно отдаёт указатель на массив, но мутация массива не защищена — будет гонка и падение. Потокобезопасность коллекции делается синхронизацией (очередь GCD, блокировка), а не атрибутом свойства.

В чём разница `id` и `instancetype`? `id` — «какой-то объект, тип неизвестен», компилятор не проверяет, что ты ему шлёшь. `instancetype` — «объект класса получателя», и компилятор это знает. Разница видна на фабричных методах и инициализаторах: если `+(instancetype)person` вернёт, компилятор понимает, что это именно `Person *`, и ловит ошибки типов; если вернуть `id`, проверки нет. Поэтому в инициализаторах и конструкторах всегда пишут `instancetype`, а не `id`.

Что такое `method swizzling` и чем он опасен? Это подмена реализации метода во время выполнения: ты меняешь местами реализации двух методов (через `method_exchangeImplementations`), и там, где система звала оригинал, теперь выполняется твой код. Применяют в аналитике, логировании, тестовых обёртках. Опасен тем, что:

- меняет поведение **глобально** — для всех экземпляров класса, включая системные;
- зависит от порядка и времени применения (где и когда сделать подмену);
- легко конфликтует с другим `swizzling` и с категориями;
- ломает интуицию при отладке: код в исходнике один, а выполняется другой.

Правильный ответ на собеседовании — «знаю, как сделать, но применяю редко и осторожно, потому что это глобальная и трудноотлаживаемая магия».

Как готовиться

Подготовка к такому собеседованию — не зубрёжка карточек. Это та самая привычка из всей книги: сомнение проверяй на живой программе.

Гоняй код в `clang` сам. Любой вопрос «что выведет» проверяется за минуту: набери пять строк, собери `clang -fobjc-arc -framework Foundation -Wall file.m -o file`, запусти, посмотри вывод. Не верь на слово ни учебнику, ни себе — верь компилятору. Память так укладывается намертво: ты не помнишь правило, ты помнишь, как оно повело себя у тебя на экране.

Читай заголовки SDK. Файлы `.h` системных фреймворков — самый честный источник о том, что класс умеет. В Xcode по `*`-клику на имя класса проваливаешься прямо в заголовок и видишь все методы, свойства, атрибуты, аннотации `nullable`. Привыкни туда заглядывать: половина «каверзных» вопросов снимается одним взглядом на объявление.

Проговаривай вслух. Это отдельный навык. Знать ответ и **сказать** его связно под взглядом человека — разные вещи. Возьми список ловушек выше и объясни каждую вслух, будто человеку напротив, без подглядывания. Споткнулся — значит, понимаешь хуже, чем казалось. Запишись на диктофон, послушай: вода и «эээ» сразу слышны.

Сверяйся с документацией Apple. Учебники устаревают, форумы врут, официальная документация ближе всего к истине. Встретил спорное утверждение про tagged pointer, autorelease или swizzling — проверь его на developer.apple.com, а не запоминай чужую байку. На собеседовании ценится не «я где-то читал», а «я знаю первоисточник».

Как разбирать задачи в этом разделе

Разделы D, E и F устроены одинаково, и пользоваться ими нужно правильно, иначе пользы не будет.

Сначала сам, потом решение. Перед каждой задачей закрой решение и дай себе настоящую попытку. Напиши код, прогони его в clang, ошибись, почини. Только потом открывай разбор и сравнивай. Если сразу читать ответ, мозг кивает «логично» и ничего не запоминает — это иллюзия понимания, которая разваливается на собеседовании. Польза рождается в момент, когда ты **застрял и выкарабкался сам**.

Где лежат решения. Задачи с однозначным, корректным решением вынесены отдельными файлами в папку code/ — их можно собрать и запустить ровно как код глав книги. В разборе задачи указано имя файла. Сначала попробуй написать своё, потом сверься с эталоном: важны не только результат, но и стиль — именование, атрибуты свойств, обработка nil.

Каверзы с UB живут только в тексте. Часть вопросов специально про **неопределённое поведение** (undefined behavior) и про то, чего делать нельзя: сообщение освобождённому объекту, опора на retainCount, гонка данных, мутация коллекции во время перебора. Такой код мы **не** кладём в code/ и не предлагаем «запустить и посмотреть» — потому что у неопределённого поведения нет «правильного вывода»: на твоей машине оно может случайно сработать, на собеседовании — упасть, а в проде — испортить данные. Эти задачи разбираем словами: почему так нельзя, что произойдёт в худшем случае, как написать правильно. Если очень хочется увидеть падение своими глазами — это нормально, но держи в голове, что увиденный результат ничего не доказывает, кроме того, что так делать нельзя.

Напоследок

Собеседование по Objective-C — это не экзамен на память, а разговор о том, понимаешь ли ты платформу. Ты уже прошёл всю книгу: от первой переменной до того, как `objc_msgSend` находит метод во время выполнения. Тебе есть что сказать.

Не пытайся выглядеть тем, кто знает всё, — таких не бывает. Гораздо сильнее звучит честное «вот это знаю и могу объяснить, а вот тут не уверен, проверил бы в коде». Именно так работает хороший разработчик: не угадывает, а проверяет. Эту привычку мы тренировали всю книгу — она и есть твоё главное преимущество в комнате.

Открывай раздел D и начинай с джуновских задач, даже если метишь выше: основы должны отлетать от зубов, иначе на глубине ты будешь спотыкаться о мелочи. Пиши код, ошибайся, проговаривай вслух. Удачи — ты готов лучше, чем тебе кажется.

Документация Apple

Ключевые первоисточники, по которым стоит готовиться. Это та же стартовая полка, что в конце книги, — но отобранная под собеседование.

Язык и идиомы:

- Programming with Objective-C — developer.apple.com → «Programming with Objective-C» (объекты, сообщения, свойства, протоколы, категории, блоки; база для Junior/Middle-вопросов).
- Coding Guidelines for Cocoa — developer.apple.com (именование методов и классов так, как этого ждут на собеседовании про проектирование API).

Память:

- Transitioning to ARC Release Notes — developer.apple.com (что делает ARC, `__strong`/`__weak`/`__unsafe_unretained` циклы удержания — основа Middle-вопросов про память).
- Advanced Memory Management Programming Guide — developer.apple.com (полная модель владения: `retain`/`release`/`autorelease`, пулы автоосвобождения).

Runtime (Senior):

- Objective-C Runtime Programming Guide — developer.apple.com (Messaging, Dynamic Method Resolution, Message Forwarding — то, что спрашивают про `objc_msgSend` и пересылку).
- Objective-C Runtime (API reference) — developer.apple.com/documentation/objectivec (`object_getClass`, `class_getName`, `method_exchangeImplementations`, `sel_getName` — для вопросов про интроспекцию и swizzling).

Многопоточность:

- Dispatch (Grand Central Dispatch) — developer.apple.com/documentation/dispatch (очереди, `dispatch_async`, барьеры, `dispatch_once` — потокобезопасность и синглтон).
- Concurrency Programming Guide — [developer.apple.com](https://developer.apple.com/concurrency/) (очереди против потоков, общая модель).

Коллекции и значения:

- Foundation — developer.apple.com/documentation/foundation (корень справочников по классам).
- Collections Programming Topics — developer.apple.com/library (массивы, словари, множества, копирование, `NSCopying`).

Interop со Swift:

- Swift and Objective-C in the Same Project — [developer.apple.com](https://developer.apple.com/swift/) (как два языка видят друг друга, аннотации `nullability`, облегчённые дженерики).

Приложение D. Задачи собеседования: Junior

Этот раздел — тренажёр. Здесь собраны вопросы, которые реально задают на собеседованиях уровня Junior по Objective-C: про синтаксис, объекты, базовую память и коллекции. Ничего нового по сравнению с основными главами тут нет — это проверка того, уложилось ли уже пройденное.

Формат у каждой задачи один. Сначала — условие. Если это вопрос «что выведет программа», не беги сразу к ответу: закрой нижнюю часть рукой, проговори ответ вслух или запиши, и только потом читай разбор. Так проверка работает, а простое чтение — нет.

Многие задачи сопровождаются компилируемым файлом в code/. Собрать и запустить любой можно одной командой:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/junior-01-nil.m -o /tmp/t && /tmp/t
```

Весь показанный вывод — настоящий, снят с этой команды. Дата, время и числа в скобках у тебя будут свои — их добавляет NSLog (см. главу 1). Часть задач намеренно про падения программы (выход за границу массива, addObject:nil). Такой код мы разбираем словами, но в code/ не кладём — чтобы проверка примеров оставалась зелёной.

Задача 1. Что выведет [nil length]

Дан код:

```
NSString *s = nil;
NSUInteger len = [s length];
NSString *up = [s uppercaseString];
```

```
B00L      has = [s hasPrefix:@"a"];
NSLog(@"%lu / %@ / %d", (unsigned long)len, up, has);
```

Программа упадёт? Если нет — что напечатает?

Разбор. Программистов из Си здесь подлавливают чаще всего. В Си проход по нулевому указателю — мгновенный крах (segmentation fault). Кажется, что и тут [nil length] обязано уронить программу. Не уронит.

В Objective-C **отправка сообщения nil — безопасная пустая операция**. Runtime видит, что получатель nil, и просто ничего не делает. А в ответ отдаёт «нулевой» результат — какой именно, зависит от типа возврата метода:

- метод возвращает число (NSUInteger, NSInteger, int...) → получишь 0;
- метод возвращает объект (NSString *, id...) → получишь nil;
- метод возвращает B00L → получишь N0.

Поэтому вывод (файл code/junior-01-nil.m):

```
[nil length]      = 0
[nil uppercaseString] = (null)
[nil hasPrefix:@"a"] = 0
```

(null) — это так NSLog печатает nil-объект. 0 в конце — это N0.

Это удобно (не нужно обвешивать каждый объект проверкой if (x != nil)), но и опасно: если объект случайно оказался nil, ошибка не выстрелит на месте, а тихо даст 0/nil/N0, и баг всплывёт позже и не там. Где результат важен — проверяй if (x == nil) явно.

Отличие от Си. *p при p == NULL в Си роняет программу. [nil message] в Objective-C — узаконенная пустая операция с нулевым результатом. Это сознательное решение языка, а не случайность.

Задача 2. == или isEqualToString:

Что напечатает каждая строка?

```
NSString *a = @"hello";
NSString *b = [NSString stringWithFormat:@"%s%s", @"hel", @"lo"];
NSLog(@"%d", (a == b));
NSLog(@"%d", [a isEqualToString:b]);
```

Разбор. Текст у a и b одинаковый — hello. Но вопрос не в тексте.

- `a == b` сравнивает **указатели**: «это один и тот же объект в памяти?».
- `[a isEqualToString:b]` сравнивает **содержимое**: «одинаковый ли текст, буква за буквой?».

`a` — это строковый литерал, он живёт в неизменяемой области программы. `b` собран в рантайме методом `stringWithFormat`: — это **другой** объект, в другом месте памяти. Значит, адреса разные, а текст одинаковый:

```
a == b                : 0
[a isEqualToString:b]: 1
адрес a = 0x10e7b1038
адрес b = 0xd146d585ee0ba9ab
```

(Адреса у тебя будут свои, важно одно — они разные.) Вывод файла `code/junior-02-string-equal.m`.

Правило на всю жизнь: **строки сравнивай по содержимому через `isEqualToString`, никогда через `==`**. Две внешне одинаковые строки `==` легко сочтёт разными. `==` для объектов уместно только в одном случае — проверить, не `nil` ли это (там мы и правда сравниваем сам факт наличия объекта, а не текст).

Отличие от Си. В Си строки тоже не сравнивали через `==` (это сравнило бы адреса `char *`) — для содержимого был `strcmp`. Здесь та же логика, только вместо `strcmp` — сообщение `isEqualToString`.

Задача 3. Два `NSNumber` и `==`

```
NSNumber *a = @42;
NSNumber *b = @42;
NSNumber *x = [NSNumber numberWithDouble:12345.6789];
NSNumber *y = [NSNumber numberWithDouble:12345.6789];
NSLog(@"%d %d", (a == b), [a isEqualToString:b]);
NSLog(@"%d %d", (x == y), [x isEqualToString:y]);
```

Что выведет?

Разбор. Логика та же, что у строк: содержимое чисел сравнивает `isEqualToNumber`, а `==` сравнивает указатели. Но с числами есть ловушка хитрее. Реальный вывод (`code/junior-03-number-equal.m`):

```
@42 == @42          : 1
[@42 isEqualToNumber:@42] : 1
x == y              : 0
[x isEqualToNumber:y]  : 1
```

Смотри: @42 == @42 дало 1 (правда!), а x == y для двух одинаковых дробных — 0 (ложь!). Как так?

Маленькие целые NSNumber рантайм на 64-битных системах хранит **прямо внутри указателя** (это называется tagged pointer — «помеченный указатель»): само число 42 закодировано в биты «адреса», отдельный объект в памяти не создаётся. Поэтому два @42 получают одинаковый «адрес», и == случайно оказывается прав. А 12345.6789 так не упаковывается — создаются два разных объекта, и == честно возвращает 0.

Вывод: == для чисел **иногда случайно прав, иногда врёт** — на него нельзя полагаться вообще. Сравнивай значения через isEqualToNumber: (a isEqualToNumber: верен в обоих случаях — обе строки с ним дали 1).

Задача 4. Что делают alloc и init и что будет, если забыть init

```
@interface Box : NSObject
@property (nonatomic) NSInteger value;
@end

@implementation Box
- (instancetype)init {
    self = [super init];
    if (self) { _value = 100; }
    return self;
}
@end

// ...
Box *raw = [Box alloc];
Box *ok  = [[Box alloc] init];
```

Чему равны raw.value и ok.value?

Разбор. Объект в Objective-C рождается в два шага, и у каждого своя работа:

- `alloc` (`allocate`, выделить) — просит у системы память под новый объект и **обнуляет** все его поля. Объект уже существует, но «сырой»: настройка ещё не выполнена.
- `init` (`initialize`, инициализировать) — приводит объект в рабочее состояние. Здесь `init` задаёт `_value = 100` (значение по умолчанию).

Если `init` забыть, останется сырой объект с нулевыми полями. Вывод (`code/junior-04-alloc-init.m`):

```
после alloc:      value = 0
после alloc + init: value = 100
```

`raw` прошёл только `alloc` → `value` нулевой. `ok` прошёл `alloc` и `init` → `value` равен 100, как задумано в инициализаторе. Поэтому почти всегда их пишут слитной парой: `[[Box alloc] init]`.

Разбери и сам инициализатор — это шаблон, который встречается постоянно:

- `self = [super init];` — сначала даём родителю настроить свою часть объекта и **присваиваем результат `self`** (родитель имеет право вернуть другой объект или `nil`);
- `if (self) { ... }` — настраиваем свои поля, только если родитель не отдал `nil`;
- `return self;` — обязательно возвращаем готовый объект.

Частый баг новичка — забыть `return self;` или `self = [super init]`. Тогда инициализатор вернёт мусор, и объект придёт битым.

Отличие от Си. В Си это было `malloc(sizeof(...))` плюс ручное заполнение полей. Здесь `alloc` выделяет память, а `init` заполняет — но заполняет сам объект, по своему чертежу. Освобождать руками не надо: память объектов считает ARC (глава 13).

Задача 5. `strong` против `copy` у свойства `NSString`

Дан класс с двумя свойствами-строками:

```
@property (nonatomic, strong) NSString *strongName;
@property (nonatomic, copy)   NSString *copiedName;
```

И код:

```
NSMutableString *m = [NSMutableString stringWithString:@"Анна"];
p.strongName = m;
p.copiedName = m;
[m appendString:@" Каренина"];
NSLog(@"%@ / %@", p.strongName, p.copiedName);
```

Что напечатает?

Разбор. Подвох в том, что `NSString` * может на самом деле указывать на `NSMutableString` — изменяемую строку (`NSMutableString` — потомок `NSString`). Мы кладем в оба свойства **одну и ту же** изменяемую строку `m`, а потом меняем `m`.

- `strong` хранит **ту же ссылку**, что ему дали. Свойство и переменная `m` указывают на один объект. Изменили `m` — изменилось и то, что видит `strongName`.
- `copy` при присваивании делает **снимок** — неизменяемую копию. Свойство получает свой отдельный объект @"Анна", на который последующие правки `m` уже не влияют.

Вывод (code/junior-05-copy-property.m):

```
strong: Анна Каренина
copy:   Анна
```

`strongName` «поехал» вслед за `m`, а `copiedName` остался снимком. Вот почему **строковые свойства почти всегда объявляют с `copy`**: ты хочешь своё неизменное значение, а не чужую строку, которую кто-то снаружи поменяет у тебя за спиной. То же правило — для `NSArray`, `NSDictionary`, `NSSet`: их свойства тоже делают `copy`.

Задача 6. BOOL, YES, NO и ловушка с числом

Что выведет?

```
NSLog(@"%d %d", YES, NO);
int weird = 256;
BOOL fromInt = (BOOL)weird;
NSLog(@"%d", fromInt);
```

Разбор. `BOOL` — это objс-шный «да/нет» с двумя именованными значениями: `YES` (под капотом 1) и `NO` (под капотом 0). Печатаем их через `%d`, потому что это маленькие целые. Первая строка очевидна: 1 0.

А вот вторая — классическая ловушка. `BOOL` — это всего один байт. Число 256 в один байт не влезает: его младший байт равен нулю ($256 = 0 \times 100$). При укладке в `BOOL` старшие биты отбрасываются, остаётся 0, то есть `NO`. Вывод (code/junior-06-bool.m):

```
YES = 1, NO = 0
sizeof(BOOL) = 1 байт
есть элементы: 1
BOOL из числа 256 = 0
```

Число было ненулевым (256, «истина по смыслу»), а BOOL получился 0, «ложь». Отсюда два правила:

1. **Не заталкивай произвольное число прямо в BOOL.** Если хочешь «не ноль ли это», пиши явное сравнение: `BOOL ok = (count > 0);`.
2. **Не сравнивай BOOL с YES напрямую** (`if (x == YES)`). Если в `x` как-то попало, скажем, 2, то `x` — истина в условии `if (x)`, но `x == YES` (то есть `2 == 1`) даст ложь. Пиши просто `if (x)` — это проверяет «истинность», а не точное равенство единице.

(Точное поведение BOOL зависит от платформы: где-то это `signed char`, где-то настоящий `bool`. Но привычка «не пихать числа в BOOL и не сравнивать с YES» спасает везде.)

Задача 7. `addObject:nil` — что произойдёт

```
NSMutableArray *a = [NSMutableArray array];
NSString *name = nil;
[a addObject:name];
```

Программа продолжит работать?

Разбор. Нет — она **упадёт** прямо на `addObject:`. И это надо чётко отделять от задачи 1.

Сообщение `nil` (получатель — `nil`) безопасно. Но здесь получатель — это живой массив `a`, а `nil` идёт **аргументом**. `NSMutableArray` не умеет хранить «ничего»: коллекции Foundation хранят только настоящие объекты. Получив `nil` аргументом, метод выбрасывает исключение вроде:

```
*** -[__NSArrayM insertObject:atIndex:]: object cannot be nil
```

и программа аварийно завершается. Тот же эффект даст литерал с `nil` внутри: `@[a, nil, b]` — крах на месте сборки массива.

Поэтому перед добавлением проверяй, что объект есть:

```
if (name != nil) {
    [a addObject:name];
}
```

А если нужно осмысленное «здесь значения нет» — кладут не `nil`, а объект-заглушку `NSNull` (см. задачу 9).

Запоминай разницу: **`nil`-получатель — тихо и безопасно; `nil`-аргумент у коллекции — краш.**

Задача 8. objectAtIndex: против firstObject на пустом массиве

```
NSArray *empty = @[];
id a = [empty firstObject];
id b = [empty objectAtIndex:0];
```

Что вернёт каждая строка?

Разбор. Они ведут себя **по-разному**, и в этом вся соль.

- firstObject (и lastObject) на пустом массиве **возвращает nil** — безопасно, без падения.
- objectAtIndex:0 на пустом массиве **роняет программу**: индекс 0 за границей пустого массива, runtime выбрасывает исключение NSRangeException — index 0 beyond bounds for empty array.

Поэтому вторую строку в файл мы не кладём. Безопасную часть проверяем в code/junior-07-firstobject.m:

```
firstObject пустого: (null)
lastObject  пустого: (null)
firstObject:      10
objectAtIndex:0: 10
lastObject:       30
```

На непустом массиве firstObject и objectAtIndex:0 дают одно и то же. Разница только на пустом: один отдаёт nil, другой падает.

Практический вывод: когда не уверен, что в массиве есть элементы, бери крайние через firstObject/lastObject — они прощают пустоту. objectAtIndex: используй, только когда индекс точно в пределах 0 ... count-1.

Задача 9. nil-значение в словаре и NSNull

```
NSMutableDictionary *d = [NSMutableDictionary dictionary];
d[@"name"] = @"Анна";
d[@"middle"] = [NSNull null];
d[@"city"] = nil;
```

```
NSLog(@"%@ / %@ / %@ / %@",
      d[@"name"], d[@"middle"], d[@"city"], d[@"absent"]);
```

Что выведет — и не упадёт ли строка с `d[@"city"] = nil`?

Разбор. Сразу про `d[@"city"] = nil` — это **не** краш и **не** вставка пустого значения. Для изменяемого словаря присваивание `nil` по ключу через скобки означает **удаление ключа**. После этой строки ключа `city` в словаре просто нет.

Хранить `nil` как значение словарь не может (та же причина, что у массива в задаче 7). Когда надо явно сказать «значение здесь отсутствует, и это осмысленно», кладут объект-заглушку `NSNull` — это настоящий объект, синглтон, который получают сообщением `[NSNull null]`.

Вывод (`code/junior-08-nsnull.m`, без строки `city`, чтобы показать работающий результат):

```
name:   Анна
middle: <null>
absent: (null)
middle — это NSNull, а не настоящее значение
```

Тонкость: `<null>` (в угловых скобках) — это как печатается объект `NSNull`. А `(null)` (в круглых) — это `nil`, то есть «ключа нет вообще» (`absent` мы не клали). Отличать их важно: за отсутствующим ключом словарь возвращает `nil`, а `NSNull` — это лежащее по ключу реальное значение, которое надо проверять сравнением `== [NSNull null]`.

Зачем вообще `NSNull`? Чаще всего он прилетает из разобранного JSON: поле с `null` в ответе сервера превращается именно в `NSNull`, и забыть про это — типичный источник багов («думал, там строка, а там `NSNull`»).

Задача 10. Почему нельзя `[array addObject:5]`

```
NSMutableArray *a = [NSMutableArray array];
[a addObject:5];           // ???
[a addObject:@5];         // ???
```

Какая строка скомпилируется, какая нет, и почему?

Разбор. Первая строка **не компилируется**. `addObject:` ждёт объект (`id`), а `5` — это голый `int`, примитив языка Си, не объект. Объекту коллекции дать примитив нельзя.

Чтобы число попало в коллекцию, его сначала **упаковывают** (`boxing`) в объект `NSNumber`. Делают это литералом:

- @5 — упаковать целое 5;
- @3.14 — упаковать дробное;
- @(10 + 5) — упаковать значение **выражения** (скобки обязательны, если внутри не одна константа, а вычисление или переменная: @(x)).

Вторая строка (@5) поэтому в порядке. Достать число обратно (unboxing) — сообщением вроде intValue / doubleValue. Вывод code/junior-09-nsnumber.m:

```
храним объект:    42
распаковали int:  42
второй элемент:   15
double обратно:   3.14
```

Отличие от Си. В Си массив `int a[]` хранит числа напрямую — это ведь просто кусок памяти. `NSMutableArray` хранит **только объекты**, поэтому число приходится заворачивать в `NSNumber`. Зато один массив может держать вперемешку строки, числа и что угодно — все они объекты.

Задача 11. Быстрый перебор for-in и мутация во время него

```
NSMutableArray *nums = [@[1, 2, 3, 4] mutableCopy];
for (NSNumber *x in nums) {
    if (x.intValue % 2 == 0) {
        [nums removeObject:x]; // удаляем прямо в цикле
    }
}
```

Что случится?

Разбор. Программа **упадёт**. Цикл `for (x in nums)` — это быстрый перебор (fast enumeration). Он проходит коллекцию и одновременно следит, не менялась ли она. Если изменить коллекцию **прямо во время перебора** (`removeObject:`, `addObject:`), runtime ловит это и выбрасывает исключение:

```
*** Collection <__NSArrayM> was mutated while being enumerated.
```

Менять то, что сейчас перебираешь, нельзя. Правильных приёмов два:

1. **Собрать кандидатов, удалить потом.** Проходим коллекцию только на чтение, копим в отдельный массив, что надо убрать, и удаляем уже после цикла.
2. Перебирать **копию** (`for (x in [nums copy])`), а менять оригинал.

Первый приём проверяем в `code/junior-10-forin.m`:

```
фрукт: яблоко (длина 6)
фрукт: банан (длина 5)
фрукт: вишня (длина 5)
без чётных: (
    1,
    3,
    5
)
```

(яблоко — 6 символов: перебор отработал штатно, потому что массив `fruits` мы только читали.) Чётные удалились корректно, потому что `removeObjectsInArray`: вызван **после** цикла, а не внутри.

Отличие от Си. Перебор `for (int i=0; i<n; i++) a[i]` в Си про мутацию ничего не знает — массив фиксированный, «сломать перебор вставкой» там нечем. Foundation-коллекции растут и сжимаются, поэтому язык специально защищается от изменения во время обхода.

Задача 12. Полиморфизм: какой метод вызовется

```
@interface Animal : NSObject
- (NSString *)sound;
- (void)describe;
@end

@implementation Animal
- (NSString *)sound { return @"..."; }
- (void)describe {
    NSLog(@"%@ говорит: %@", NSStringFromClass([self class]), [self sound]);
}
@end

@interface Dog : Animal @end
@implementation Dog
- (NSString *)sound { return @"Гав"; }
@end

// аналогично Cat -> @"Мяу"
```

```
NSArray *zoo = @[[Animal new], [Dog new], [Cat new]];
for (Animal *a in zoo) { [a describe]; }
```

describe написан **один раз** в Animal. Что напечатают три вызова?

Разбор. Здесь работает полиморфизм. Переменная цикла объявлена как Animal *a, но это лишь обещание компилятору «тут будет животное». А какой метод реально выполнится, runtime выбирает по **фактическому классу** объекта в момент выполнения (динамическая диспетчеризация, глава 5).

describe унаследован всеми и сам по себе не переопределён — поэтому выполняется один и тот же его код. Но внутри он шлёт [self sound], и вот тут self — это конкретный объект: для Dog сработает Dog-овский sound, для Cat — кошачий. Вывод (code/junior-11-polymorphism.m):

```
Animal говорит: ...
Dog говорит: Гав
Cat говорит: Мяу
```

Один метод describe, а звук у каждого свой — потому что sound выбирается по реальному классу self, а не по типу переменной a. NSStringFromClass([self class]) подтверждает: класс у каждого свой.

Отличие от Си. В Си describe(animal) — обычная функция, она вызвала бы один и тот же sound, какой указали при компиляции. Здесь объект сам решает в рантайме, какой sound его — это и есть «отправка сообщения», а не «вызов функции».

Задача 13. isKindOfClass: против isKindOfClass:

Dog — потомок Animal. Что выведет?

```
Dog *d = [Dog new];
NSLog(@"%d", [d isKindOfClass:[Animal class]]);
NSLog(@"%d", [d isKindOfClass:[Animal class]]);
NSLog(@"%d", [d isKindOfClass:[Dog class]]);
```

Разбор. Оба сообщения спрашивают объект о его классе, но по-разному:

- isKindOfClass: — «объект относится к этому классу **или любому его потомку?**». Учитывает всю цепочку наследования.
- isKindOfClass: — «объект относится **ровно** к этому классу?». Точное совпадение, потомки не считаются.

d — это Dog, а Dog — потомок Animal. Значит:

- [d isKindOfClass:[Animal class]] → YES (Dog — разновидность Animal);
- [d isKindOfClass:[Animal class]] → NO (d не равно Animal, а Dog);
- [d isKindOfClass:[Dog class]] → YES (вот теперь точное совпадение).

Полный вывод code/junior-12-isKindOfClass.m:

```
isKindOfClass:Dog      = 1
isKindOfClass:Animal   = 1
isKindOfClass:NSObject = 1
isKindOfClass:Dog      = 1
isKindOfClass:Animal   = 0
```

isKindOfClass:NSObject тоже 1 — NSObject корень всех классов, любой объект ему «родственник». На практике почти всегда нужен именно isKindOfClass: (он учитывает наследование). isKindOfClass: берут редко — когда важно поймать строго один класс и отсеять потомков.

Задача 14.instancetype или id в фабричном методе

В чём разница между двумя объявлениями и почему первое лучше?

```
+ (instancetype)widgetWithTitle:(NSString *)title;    // так
+ (id)widgetWithTitle:(NSString *)title;              // или так?
```

Разбор. Оба варианта вернут объект, и программа в обоих случаях заработает. Разница в том, **что знает компилятор** о типе результата.

- id — «какой-то объект, тип неизвестен». Компилятор не проверит, что ты потом с ним делаешь.
- instancetype — «объект **именно этого** класса» (того, у которого вызвали метод). Компилятор знает точный тип и проверит обращения к нему.

Зачем это нужно на практике:

```
Widget *w = [Widget widgetWithTitle:@"Кнопка"];
w.title;   // с instancetype — компилятор знает про .title, всё ок
```

С instancetype тип w выводится как Widget *, и .title доступно сразу. С id компилятор не знал бы класс точно, и проще нарваться на ошибку или потерять автодополнение и проверки. Особенно это важно при наследовании: фабричный метод с instancetype, написанный в Widget, для подкласса RoundWidget вернёт уже RoundWidget *, а с id — просто «объект».

Внутри метода для этого пишут `[[self alloc] init]`, а не `[[Widget alloc] init]:self` в методе класса — это сам класс, по которому метод вызвали, поэтому подкласс получит объект своего класса. Проверка `code/junior-13-instancetype.m`:

```
создан Widget с title = Кнопка
```

Правило: для `alloc/init` и фабричных методов («конструкторов») возвращай **`instancetype`**. `id` оставляй там, где тип результата и правда заранее не известен.

Задача 15. Реализуй стек поверх `NSMutableArray`

Напиши класс `Stack` с операциями `push:`, `pop`, `peek`, `isEmpty`, `count`. Стек — это «стопка тарелок»: кладём сверху и снимаем сверху (LIFO, last in — first out).

Разбор. Стек удобно построить поверх готового `NSMutableArray`, работая только с **одним** его концом. Берём конец массива за «верх стопки»: `addObject:` кладёт в конец, `lastObject` смотрит конец, `removeLastObject` снимает с конца. Все три операции — быстрые.

```
@interface Stack : NSObject
- (void)push:(id)object;
- (id)pop;
- (id)peek;
- (BOOL)isEmpty;
- (NSUInteger)count;
@end

@implementation Stack {
    NSMutableArray *_items;          // приватное хранилище
}

- (instancetype)init {
    self = [super init];
    if (self) { _items = [NSMutableArray array]; }
    return self;
}

- (void)push:(id)object { [_items addObject:object]; }

- (id)pop {
```

```
    if (_items.count == 0) return nil;    // защита от выхода за границу
    id top = [_items lastObject];
    [_items removeLastObject];
    return top;
}

- (id)peek    { return [_items lastObject]; }    // на пустом вернёт nil
- (BOOL)isEmpty { return _items.count == 0; }
- (NSUInteger)count { return _items.count; }

@end
```

Главное в реализации:

- Хранилище `_items` объявлено в фигурных скобках `@implementation` — это приватное поле, снаружи его не видно. Класс отдаёт наружу только осмысленные операции, а на чём он построен — его дело (инкапсуляция).
- В `pop` стоит проверка на пустоту: `removeLastObject` у пустого массива упал бы, поэтому сначала возвращаем `nil`. `peek` через `lastObject` проверки не требует — `lastObject` на пустом сам отдаёт `nil`.
- `_items` создаём в `init`, иначе он останется `nil`, и `push`: будет тихо слать сообщения в `nil` (см. задачу 1) — элементы «добавлялись» бы в никуда.

Вывод `code/junior-14-stack.m`:

```
пустой в начале: 1
после трёх push, count = 3
peek (не снимая): c
pop: c
pop: b
после двух pop, count = 1
pop: a
pop пустого: (null)
пустой в конце: 1
```

Видно главное свойство LIFO: положили `a`, `b`, `c`, а снимаются они в обратном порядке — `c`, `b`, `a`. `pop` пустого стека отдал `nil` и не уронил программу.

Задача 16. Найди утечку: сильная ссылка на delegate

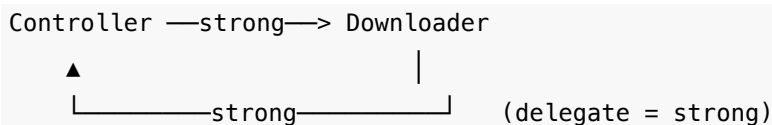
Что не так с этим кодом и к чему приведёт?

```
@interface Downloader : NSObject
@property (nonatomic, strong) id<DownloadDelegate> delegate; // strong
@end

@interface Controller : NSObject <DownloadDelegate>
@property (nonatomic, strong) Downloader *downloader;
@end

// контроллер создаёт загрузчик и ставит себя его делегатом:
self.downloader = [Downloader new];
self.downloader.delegate = self;
```

Разбор. Здесь **цикл удержания** (retain cycle) — обе стороны держат друг друга сильно, и ни одна не может освободиться:



Controller сильно держит Downloader (свойство downloader). Downloader сильно держит Controller обратно (свойство delegate). Когда Controller должен бы умереть, его всё ещё держит Downloader — а Downloader держит Controller. ARC видит, что у обоих ещё есть владелец, и не освобождает ни того, ни другого. Память утекла, dealloc не вызовется никогда.

Лечение — сделать delegate слабой ссылкой (weak):

```
@property (nonatomic, weak) id<DownloadDelegate> delegate;
```

weak указывает на объект, но **не удерживает** его. Цикл разрывается: сильная связь остаётся только одна (Controller → Downloader), и когда контроллер уходит, освобождаются оба. Файл code/junior-15-delegate.m доказывает это печатью из dealloc:

```
Controller: загрузка завершена
Controller освобождён
Downloader освобождён
после блока — оба объекта уже мертвы
```

Оба dealloc сработали — утечки нет. Будь delegate сильным, этих двух строк про «освобождён» в выводе не было бы вовсе.

Правило, которое спрашивают почти на каждом собеседовании: **свойство delegate всегда weak**. Делегат — это «тот, кому я докладываю», а не «тот, кем я владею»; владеть им я не должен.

Отличие от Си. В Си памятью управляешь сам (malloc/free), и «циклов удержания» нет — есть просто забытый free (утечка) или двойной free (крах). Здесь память считает ARC по числу сильных ссылок, и новый класс ошибок — два объекта, держащие друг друга. weak — штатное средство такие циклы рвать.

Задача 17. mutableCopy и copy — что можно менять

```
NSArray *a = @[1, 2, 3];
NSMutableArray *m1 = [a copy];
NSMutableArray *m2 = [a mutableCopy];
[m2 addObject:@4]; // строка A
[m1 addObject:@5]; // строка B
```

Какая строка упадёт?

Разбор. Подвох в типе, который реально возвращают эти сообщения, а не в том, что написано слева.

- copy у массива возвращает **неизменяемый** NSArray — даже если ты присвоил его переменной типа NSMutableArray *. Компилятор поверит объявлению и пропустит код, но в рантайме объект-то неизменяемый.
- mutableCopy возвращает **изменяемый** NSMutableArray — настоящую редактируемую копию.

Поэтому:

- строка A ([m2 addObject:@4]) — в порядке: m2 действительно изменяемый.
- строка B ([m1 addObject:@5]) — **краш**: m1 на самом деле неизменяемый NSArray, и в ответ на addObject: прилетит исключение unrecognized selector/mutating method sent to immutable object.

Тип переменной (NSMutableArray *) — это лишь обещание компилятору, а изменяемость определяет **реальный объект**, который вернуло сообщение. copy всегда отдаёт неизменяемый вариант, mutableCopy — изменяемый, как бы ты ни назвал переменную. Этот код мы не кладём в code/, потому что строка B роняет программу.

Задача 18. Объект-значение как ключ словаря

```
NSMutableDictionary *d = [NSMutableDictionary dictionary];
NSString *k1 = [NSString stringWithFormat:@"user_%d", 7];
d[k1] = @"Анна";
NSString *k2 = [NSString stringWithFormat:@"user_%d", 7];
NSLog(@"%@", d[k2]);
```

k1 и k2 — разные объекты (см. задачу 2). Найдётся ли значение по k2?

Разбор. Да, напечатает Анна. И это важно понять: словарь ищет ключ **не по адресу, а по содержимому**.

Когда кладёшь и достаёшь по ключу, NSDictionary не сравнивает указатели через ==. Он спрашивает у ключа две вещи: его hash (число-отпечаток) и isEqual: (равенство по содержимому). У двух строк user_7 хеш одинаковый и isEqual: вернёт YES, хотя это разные объекты в памяти. Поэтому ключ находится.

Отсюда следствие, о котором спрашивают чуть выше уровнем: если делаешь **свой** класс ключом словаря, ты обязан правильно реализовать у него пару isEqual: и hash (согласованно: равные объекты — одинаковый хеш). Иначе словарь не найдёт по ключу то, что в него положил. У готовых NSString и NSNumber это уже сделано за тебя — потому строки и числа и используют ключами чаще всего.

Ещё деталь: ключи словарь **копирует** (copy). Поэтому ключом обычно берут неизменяемые объекты — NSString, NSNumber. Изменяемую строку ключом брать опасно: поменяешь её после вставки — собьётся хеш, и значение «потеряется».

Задача 19. Свойство readonly и приватная изменяемость

Снаружи класс отдаёт неизменяемый список, а внутри хочет его пополнять. Как так сделать и где тут типичная ошибка новичка?

```
@interface Playlist : NSObject
@property (nonatomic, copy, readonly) NSArray *songs;
- (void)addSong:(NSString *)song;
@end
```

Разбор. readonly означает «снаружи есть только геттер, сеттера нет». Это правильное оформление: чужой код может прочитать playlist.songs, но не подменит весь список и не полезет добавлять в него напрямую.

Ошибка новичка — отдать наружу **тот самый** внутренний NSMutableArray:

```
- (NSArray *)songs { return _songs; } // _songs изменяемый!
```

Тип в объявлении — NSArray *, но фактически вернётся изменяемый объект. Любой снаружи сможет привести его обратно к NSMutableArray и менять твой внутренний список за спиной — инкапсуляция сломана. Правильно — внутри держать NSMutableArray, а наружу отдавать **снимок**:

```
- (NSArray *)songs { return [_songs copy]; }
```

(Атрибут copy у свойства намекает на ту же идею — отдавать неизменяемую версию.) А пополнять список — только через свой метод addSong:, который работает с приватным _songs напрямую. Так класс полностью контролирует, кто и когда меняет его данные. Это та же мысль, что в задачах 5 и 17: не отдавай наружу изменяемое нутро, отдавай неизменяемый снимок.

Задача 20. Что такое self в методе и зачем [super init]

```
- (instancetype)init {
    self = [super init];
    if (self) {
        _value = 100;
    }
    return self;
}
```

Объясни каждую строку: что такое self, что такое super, почему результат [super init] присваивают обратно self.

Разбор. Это шаблон назначенного инициализатора, и его просят разобрать очень часто.

- **self** — внутри любого метода это «текущий объект-получатель», тот, кому послали сообщение. Через self объект обращается к своим полям и шлёт сообщения сам себе.
- **super** — тот же самый объект self, но «глядя на него как на родителя»: сообщение через super ищет метод не в своём классе, а начиная с **родительского**. [super init] означает «пусть сначала родитель (NSObject и выше) настроит свою часть объекта».

Почему self = [super init];, а не просто [super init];? Потому что init родителя **имеет право вернуть другой объект** или nil. Это редко, но бывает (так устроены некоторые классы Foundation). Если родитель вернул другой объект, дальше настраивать надо именно его,

поэтому результат кладут обратно в `self`. А `if (self)` страхует от случая, когда родитель вернул `nil` (инициализация не удалась) — тогда мы не лезем настраивать поля у несуществующего объекта и сразу вернём `nil`.

Порядок строго такой: **сначала родитель** (`[super init]`), **потом своя настройка** (`_value = 100`). Наоборот нельзя — пока родитель не привёл объект в порядок, трогать его рано. И в конце обязательно `return self`; — без него инициализатор вернёт мусор. Рабочий пример этого шаблона — `code/junior-04-alloc-init.m` из задачи 4.

Отличие от Си. В Си не было ни `self`, ни `super` — была функция и переданные ей аргументы. Здесь объект знает сам себя (`self`) и умеет позвать «родительскую версию» метода (`super`) — это часть объектной модели, которой в чистом Си попросту нет.

Что повторить перед собеседованием

Короткий чек-лист по этому разделу. Если на любой пункт отвечаешь не уверенно — вернись к нужной задаче.

- Сообщение `nil`-получателю безопасно и даёт `0/nil/NO`; `nil` аргументом коллекции — краш (задачи 1, 7).
- Строки и числа сравнивают по содержимому: `isEqualToString:` / `isEqualToNumber:`, `ne ==` (задачи 2, 3).
- Объект рождается связкой `[[Класс alloc] init]`; `init` строится по шаблону `self = [super init] ... return self` (задачи 4, 20).
- Строковые и коллекционные свойства объявляй с `copy`; наружу отдавай неизменяемый снимок, не внутренний `mutable` (задачи 5, 17, 19).
- `BOOL` — это `YES/NO`; не пихай в него произвольные числа и не сравнивай с `YES` напрямую (задача 6).
- Числа в коллекциях — только упакованные в `NSNumber` (`@5`, `@(x)`); отсутствие значения — `NSNull`, не `nil` (задачи 9, 10).
- Коллекцию нельзя менять во время `for-in` (задача 11).
- Метод выбирается по фактическому классу объекта — полиморфизм; `isKindOfClass:` учитывает наследование, `isMemberOfClass:` — нет (задачи 12, 13).
- Для `alloc/init` и фабрик возвращай `instancetype`, не `id` (задача 14).
- `delegate` всегда `weak` — иначе цикл удержания (задача 16).

Документация Apple

- Programming with Objective-C → Working with Objects (объекты, сообщения, nil, id) — developer.apple.com/library.
- Programming with Objective-C → Encapsulating Data (свойства, copy, strong/weak, instance-type) — там же.
- NSObject Protocol (isEqual:, hash, isKindOfClass:, isKindOfClass:, respondsToSelector:) — developer.apple.com/documentation/objectivec/nsobjectprotocol
- NSString — developer.apple.com/documentation/foundation/NSString
- NSNumber — developer.apple.com/documentation/foundation/NSNumber
- NSArray / NSMutableArray — developer.apple.com/documentation/foundation/NSArray и developer.apple.com/documentation/foundation/NSMutableArray
- NSDictionary / NSMutableDictionary — developer.apple.com/documentation/foundation/NSDictionary
- NSNull — developer.apple.com/documentation/foundation/NSNull
- Advanced Memory Management Programming Guide (ARC, сильные/слабые ссылки, циклы удержания) — developer.apple.com/library.

Приложение Е. Задачи собеседования: Middle

Junior-раздел проверял, помнишь ли ты механику: сообщения, nil, свойства, простой ARC. Middle-уровень спрашивает иначе — не «что делает эта строка», а «что произойдёт с памятью», «что выведет этот код и почему», «как ты спроектируешь вот это». Здесь почти не бывает ответа в одну строку: за каждым вопросом стоит модель владения, поведение runtime или контракт класса.

Формат тот же, что в Junior-разделе. Каждая задача — это **условие**, потом **подумай сам** (остановись и попробуй ответить), потом **разбор** по правилам Objective-C и, где задача про код, — **компилируемое решение** в code/middle-NN-имя.m. Каждый показанный вывод реальный: файлы собраны и запущены, dealloc-логи настоящие. Дата, имя программы и число в квадратных скобках, которые NSLog печатает в начале строки, здесь убраны — у тебя они будут свои.

Собрать и запустить любой файл:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/middle-01-retain-cycle.m -o /tmp/t && /tmp/t
```

Идём от простого к сложному: сперва блоки и память, потом копирование, протоколы и категории, в конце — KVC/KVO и контракты NSCopying, isEqual:/hash.

Задача 1. Что выведет блок: захват по значению и __block

Условие. Перед тобой три фрагмента. Скажи, что напечатает каждый.

```
int x = 10;
void (^plain)(void) = ^{ NSLog(@"%d", x); };
x = 99;
```

```

plain();                                // (1) ?

__block int y = 10;
void (^shared)(void) = ^{ NSLog(@"%d", y); };
y = 99;
shared();                                // (2) ?

NSMutableArray *blocks = [NSMutableArray array];
for (int i = 0; i < 3; i++) {
    [blocks addObject:^( NSLog(@"%d", i); )];
}
for (void (^b)(void) in blocks) { b(); } // (3) ?

```

Подумай сам. Когда блок «снимает» значение переменной — в момент создания или в момент вызова?

Разбор. Обычный захват фиксирует **копию значения в момент создания блока** (глава 12). Дальнейшие изменения переменной блок не видит.

- (1) `plain` захватил `x` равным `10`. Присваивание `x = 99` идёт уже после создания блока — на захваченную копию оно не влияет. Печатает **10**.
- (2) `__block` меняет природу захвата: `y` становится **общей ячейкой**, на которую смотрят и внешний код, и блок. К моменту вызова `shared` в ячейке лежит `99`. Печатает **99**.
- (3) Каждая итерация создаёт новый блок и захватывает **своё** значение `i` на тот момент: `0`, `1`, `2`. Печатает **0 1 2**.

Третий пункт — частая ловушка для тех, кто пришёл из языков, где замыкание в цикле захватывает переменную «по ссылке» и все блоки потом видят финальное значение. В Objective-C без `__block` каждый блок держит свою копию. Сделай `i` объявленным как `__block` — и все три блока напечатают одно и то же финальное значение цикла.

Решение: `code/middle-02-block-capture.m`.

```

plain видит x = 10
shared видит y = 99
блок помнит i = 0
блок помнит i = 1
блок помнит i = 2

```

Задача 2. weak после смерти объекта и ловушка unsafe_unretained

Условие. Объект создан во вложенном блоке и умирает на выходе из него. Снаружи на него смотрит __weak-ссылка. Что напечатают две последние строки — и что было бы, объяви мы ссылку unsafe_unretained?

```
__weak Ticket *weakRef = nil;
@autoreleasepool {
    Ticket *t = [[Ticket alloc] init];
    t.code = @"A-17";
    weakRef = t;
} // t умирает здесь
NSLog(@"%@", weakRef); // ?
NSLog(@"%lu", (unsigned long)weakRef.code.length); // ?
```

Подумай сам. Что лежит в weakRef после смерти объекта — мусор, nil или старый адрес?

Разбор. weak — это ссылка-наблюдатель: она не удерживает объект, а когда тот умирает, ARC сам обнуляет её в nil (глава 13). Поэтому слабая ссылка либо показывает живой объект, либо честный nil — но никогда не висячий указатель.

- NSLog(@"%@", weakRef) после смерти t печатает (**null**).
- weakRef.code.length — это сообщение к nil. В Objective-C сообщение к nil безопасно и возвращает «нулевое» значение, для числа — 0.

Будь ссылка unsafe_unretained (или старое свойство assign), всё иначе: она **не обнуляется**. После смерти объекта она осталась бы указывать на освобождённую память — висячий указатель. Первая же NSLog(@"%@", ...) по нему — обращение к мёртвому объекту, неопределённое поведение: мусор или падение. Правило простое: нужна необременяющая ссылка «вверх» или на делегата — бери weak, а не unsafe_unretained, если только тебе не важна каждая наносекунда и ты не доказал, что объект переживёт ссылку.

Решение: code/middle-03-weak-nil.m.

```
объект жив: weakRef.code = A-17
dealloc Ticket (A-17)
объект мёртв: weakRef = (null)
сообщение к nil безопасно: длина = 0
```

Строка dealloc Ticket (A-17) встала ровно на закрывающей скобке вложенного блока — объект умер там, и сразу после слабая ссылка стала nil.

Задача 3. Найди утечку: команда и игроки

Условие. Команда владеет игроками (сильно держит их в массиве), а каждый игрок ссылается на свою команду обратной ссылкой. Найди ошибку, из-за которой `dealloc` не сработает, и почини её.

```
@interface Player : NSObject
@property (nonatomic, copy) NSString *name;
@property (nonatomic, strong) Team *team;    // ← подозрительно
@end

@interface Team : NSObject
@property (nonatomic, strong) NSMutableArray<Player *> *players;
@end
```

Подумай сам. Кто кого держит и где замыкается кольцо?

Разбор. Считаю сильные ссылки. `Team.players` сильно держит каждого `Player` — это правильно, команда владеет составом. Но `Player.team` объявлен `strong` — игрок тоже сильно держит команду. Получается кольцо: команда → игрок → команда. Когда снаружи на оба объекта ссылок не останется, их счётчики не дойдут до нуля, `dealloc` не сработает — классическая утечка (глава 13).

Чинить надо ту ссылку, которая **не владеет по смыслу**. Команда владеет игроком — `Team.players` оставляем `strong`. Игрок командой не владеет, лишь смотрит «вверх» — значит `Player.team` должно быть **weak**. Меняем один атрибут, и кольцо разорвано:

```
@property (nonatomic, weak) Team *team;    // вверх не владеем
```

Это та же тройка случаев, что и в главе 13: `parent ↔ child`, `объект ↔ блок`, `объект ↔ delegate`. Лечение всегда одно — одну из двух ссылок сделать слабой, и слабой делает ту, что «вверх» или «вбок», а не «вниз к собственности».

Решение: `code/middle-01-retain-cycle.m` (уже с починкой — `weak`).

```
Асет играет за Кайрат
выходим из блока — ждём три dealloc...
dealloc Team (Кайрат)
dealloc Player (Асет)
dealloc Player (Болат)
блок закрыт — dealloc были выше, утечки нет
```

Три `dealloc` на месте — утечки нет. Верни `Player.team` обратно в `strong`, пересобери — и все три строки `dealloc` исчезнут. Вот так, по пропавшему `dealloc`, утечку и ловят.

Задача 4. Цикл удержания объект ☒ блок: что выведет dealloc

Условие. Объект хранит блок в своём свойстве, а блок внутри обращается к self. Два варианта метода — какой из них утечёт и почему? Сработает ли dealloc?

```
@interface Worker : NSObject
@property (nonatomic, copy) void (^job)(void);
@property (nonatomic, copy) NSString *name;
@end

- (void)setupBad {
    self.job = ^{ NSLog(@"%@", self.name); };           // A
}

- (void)setupGood {
    __weak typeof(self) weakSelf = self;
    self.job = ^{ NSLog(@"%@", weakSelf.name); };       // B
}
```

Подумай сам. Что захватывает блок, когда упоминает self.name?

Разбор. Блок захватывает объекты **сильно**. Стоит блоку упомянуть self.name, как он захватывает сам self — сильно. А self сильно держит блок в свойстве job (атрибут copy, для блоков это правильно). Кольцо: self → job → self.

- В варианте A (setupBad) кольцо замкнуто. После того как внешний код отпустит Worker, его счётчик останется 1 (держит блок), dealloc **не сработает**. Кстати, clang это кольцо видит и предупреждает: capturing 'self' strongly in this block is likely to lead to a retain cycle [-Warc-retain-cycles]. Глушить предупреждение нельзя — это и есть баг.
- В варианте B (setupGood) блок захватывает не self, а weakSelf — слабую копию ссылки. Сильно self блок больше не держит, кольца нет. Когда внешний код отпустит Worker, тот спокойно уничтожится, и dealloc **сработает**.

Полный рабочий пример с обоими вариантами и живыми dealloc-логами уже разобран в главе 12 (code/12-retain-cycle.m): для bad строки dealloc нет, для good — есть. Идиома __weak typeof(self) weakSelf = self; — твой главный инструмент против цикла «объект ☒ блок». Если внутри блока self нужен несколько раз и важно, чтобы он не исчез посреди работы, делают ещё __strong typeof(weakSelf) strongSelf = weakSelf; в начале блока — это «weak-strong dance», но базовый ответ на собеседовании — weakSelf.

Задача 5. Зачем @autoreleasepool внутри длинного цикла

Условие. Тебе говорят: «оберни тело тяжёлого цикла в @autoreleasepool». Объясни, что это даёт. Покажи на примере, что объект, которого держит только пул, живёт ровно до слива пула.

Подумай сам. Кто освобождает временные объекты, у которых нет явного владельца, и **когда**?

Разбор. Многие объекты живут «в долг»: их никто не держит сильно, и в живых их удерживает только **пул автоосвобождения** (глава 13). Пул сливается (drain) на своей закрывающей скобке и шлёт всем накопленным объектам отложенный release. Если ты в длинном цикле плодишь тысячи таких временных объектов, они копятся в пуле до самого его слива — память пухнет, хотя на каждом шаге тебе нужен лишь один. Свой @autoreleasepool внутри тела цикла сливается на каждой итерации и держит пик памяти низким.

Важная честная оговорка про **современный ARC с -02**. По нашим экспериментам компилятор агрессивно оптимизирует возвращаемые временные объекты (return-value optimization) и часто освобождает их сразу на конце выражения, вообще не пропуская через пул. Поэтому простой цикл с [NSString stringWithFormat:] на оптимизированной сборке память почти не растит — ARC справляется и без тебя. Но как только объект реально попадает в пул (его держит только пул), правило снова работает: он живёт до слива. Чтобы показать это наглядно, в примере мы кладём объект в пул явно через CFAutorelease(CFBridgingRetain(t)) — эти функции в обход ARC-оптимизации отдают объект пулу.

Решение: code/middle-04-autoreleasepool.m.

```
=== объект живёт ровно до слива пула ===
внутри пула: Temp(1) ещё жив (dealloc не было)
    dealloc Temp(1)
после слива: Temp(1) уже мёртв (dealloc был выше)
=== тяжёлый цикл: пул на каждой итерации ===
итерация 0: объект создан
    dealloc Temp(100)
итерация 1: объект создан
    dealloc Temp(101)
итерация 2: объект создан
    dealloc Temp(102)
цикл закончен — пул не дал памяти расти
```

dealloc Temp(1) встал точно между «внутри пула» и «после слива» — объект дожил до закрывающей скобки пула и умер на ней. В цикле каждый Temp умирает на сливе своей итерации, а не копится до конца.

Задача 6. Поверхностная копия коллекции

Условие. Есть массив, внутри которого лежит **мутабельный** вложенный массив. Делаем mutableCopy и потом меняем вложенный массив. Что увидим в обеих коллекциях?

```
NSMutableArray *inner = [NSMutableArray arrayWithObjects:@"a", @"b", nil];
NSArray *original = @[inner, @"hello"];
NSMutableArray *shallow = [original mutableCopy];
[inner addObject:@"c"];
// original[0] = ?   shallow[0] = ?
```

Подумай сам. Что именно копирует mutableCopy — контейнер, его содержимое или и то и другое?

Разбор. copy и mutableCopy у коллекций делают **поверхностную копию** (shallow copy): создаётся новый контейнер, но в него кладутся **те же самые** объекты, что были в оригинале, — по ссылке, без копирования. Контейнеры теперь разные (original != shallow), а вот вложенный массив inner — **общий** на обе коллекции (original[0] == shallow[0]).

Поэтому [inner addObject:@"c"] меняет один объект, видимый из обеих коллекций: и original[0], и shallow[0] покажут (a, b, c). Копия защитила тебя только от изменений **верхнего** уровня (добавь элемент в shallow — original не дрогнет), но не от изменений вложенных мутабельных объектов.

Когда нужна полная независимость, делают **глубокую копию** (deep copy). Простой способ — прогнать структуру через архиватор: NSKeyedArchiver → NSKeyedUnarchiver. После такой копии вложенные массивы тоже становятся отдельными.

Решение: code/middle-05-shallow-copy.m.

```
разные контейнеры? да
общий вложенный массив? да
original[0] = (
    a,
    b,
    c
)
shallow[0] = (
    a,
    b,
    c
)
после deep-копии inner += d:
```

```
original[0] = (
    a,
    b,
    c,
    d
)
deep[0]      = (
    a,
    b,
    c
)
```

После `mutableCopy` оба массива видят `c`. После глубокой копии добавленный `d` ушёл только в `original` — `deep` его не заметил, потому что у него свой, отдельный вложенный массив.

Задача 7. `copy`-семантика у `NSString`-свойства

Условие. Свойство объявлено `strong`, а присваивают ему `NSMutableString`. Потом исходную строку меняют. Что станет со значением свойства? Сравни со свойством `copy`.

```
@property (nonatomic, strong) NSString *strongName;
@property (nonatomic, copy)   NSString *safeName;

NSMutableString *src = [NSMutableString stringWithString:@"Айгуль"];
p.strongName = src;
p.safeName   = src;
[src appendString:@" Сергеевна"]; // меняем ИСХОДНУЮ строку
```

Подумай сам. Что хранит свойство `strong` — снимок строки или ссылку на ту же `src`?

Разбор. `NSMutableString` — наследник `NSString`, поэтому его можно присвоить свойству типа `NSString *`. Но дальше всё зависит от атрибута:

- `strong` сохраняет **ту же ссылку** на `src`. Изменили `src` — и `strongName` «поехал» вслед, ведь это один и тот же объект. После `appendString`: он покажет Айгуль Сергеевна. Объект мутирует за спиной владельца — источник коварных багов.
- `copy` в сеттере вызывает `[src copy]` и сохраняет **неизменяемый снимок**. Дальнейшие изменения `src` его не трогают: `safeName` так и останется Айгуль.

Отсюда правило, которое на собеседовании ждут почти всегда: **строковые свойства (NSString, NSArray, NSDictionary и любые типы с мутабельным наследником) объявляй copy**. Тогда снаружи никто не подсунет тебе мутабельный объект и не изменит его потом за твоей спиной. У copy для неизменяемой строки накладных расходов почти нет — [immutableString copy] часто возвращает тот же объект с поднятым счётчиком, а не настоящую копию.

Решение: code/middle-08-string-copy.m.

до мутации:

```
strongName = Айгуль
safeName    = Айгуль
```

после мутации src:

```
strongName = Айгуль Сергеевна
safeName    = Айгуль
```

Одно и то же присваивание, разные атрибуты — и strong «протёк» изменением исходной строки, а copy устоял.

Задача 8. Простой кэш на NSMutableDictionary и copy-ключи

Условие. Напиши простой кэш «ключ-значение» поверх NSMutableDictionary. Объясни, что произойдёт, если в качестве ключа передать NSMutableString, а потом изменить эту строку. Останется ли значение доступным?

Подумай сам. NSDictionary хранит ссылку на твой ключ или его копию?

Разбор. NSDictionary хранит **копию** ключа — поэтому ключ обязан поддерживать NSCopying, а при вставке словарь сам вызывает у ключа copy. Это сделано именно ради защиты: если бы словарь держал ссылку на твой мутабельный ключ, ты бы изменил строку — изменился бы и её hash, и словарь больше не нашёл бы значение по «тому же» ключу. Снимок-копия замораживает ключ в момент вставки.

Значит, наш кэш над NSMutableDictionary уже корректен: setObject:forKey: сам копирует ключ. Передали NSMutableString @"user:1", потом дописали к ней -changed — в словаре всё равно лежит замороженная копия user:1. Значение достаётся по литералу @"user:1", а по изменённой строке — уже нет (другой ключ).

Когда кэш должен ещё и сбрасываться под нехватку памяти, вместо NSMutableDictionary берут **NSCache**: тот сам выкидывает элементы при давлении на память и потокобезопасен. Но

семантику ключей NSCache не копирует (ключ просто retain), поэтому для строковых ключей либо копируй их сам, либо используй неизменяемые.

Решение: code/middle-13-cache.m.

```
по 'user:1':      Асет
по изменённому ключу: (null)
объектов в кэше: 1
```

Значение нашлось по оригинальному ключу и не нашлось по изменённой строке — доказательство того, что словарь сохранил копию ключа, а не ссылку на твою мутабельную строку.

Задача 9. Спроектируй делегат с необязательными методами

Условие. Спроектируй протокол делегата для загрузчика: один метод обязательный («завершилось»), один необязательный («прогресс»). Покажи, как загрузчик зовёт необязательный метод безопасно, и объясни, почему свойство delegate обычно weak.

Подумай сам. Что будет, если позвать @optional-метод у делегата, который его не реализует?

Разбор. Протокол делит методы директивами @required (по умолчанию) и @optional. Обязательный метод протокол гарантирует — его можно звать без проверки. А необязательный делегат может и не реализовать; если позвать несуществующий метод напрямую — unrecognized selector, падение. Поэтому перед вызовом @optional-метода спрашивают respondsToSelector: — реализует ли делегат этот селектор:

```
if ([self.delegate respondsToSelector:@selector(downloaderDidProgress:)]) {
    [self.delegate downloaderDidProgress:0.5];
}
```

Свойство-делегат объявляют weak по той же причине, что в задаче 3: почти всегда владелец держит объект, а тот через delegate смотрит назад на владельца. Сделай delegate сильным — получишь цикл удержания владелец → объект → delegate → владелец. Слабая ссылка рвёт кольцо и заодно сама обнуляется в nil, если делегат умер раньше, — тогда вызовы к self.delegate просто молча уходят в nil.

Тип свойства — id<DownloaderDelegate>: «любой объект, лишь бы соответствовал протоколу». Протокол наследует <NSObject>, чтобы у делегата гарантированно был respondsToSelector:.

Решение: code/middle-06-delegate.m.

```
--- minimal (без optional) ---  
Minimal: готово, DATA  
--- full (с optional) ---  
Full: прогресс 50%  
Full: готово, DATA
```

Первый делегат реализует только обязательный метод — прогресс пропущен без падения. Второй реализует оба — прогресс вызван. Один и тот же загрузчик, поведение задаёт делегат.

Задача 10. Две категории с одинаковым методом — что вызовется

Условие. В двух разных категориях одного класса объявлен метод с **одинаковым** именем, но разной реализацией. Какая из них победит при вызове?

```
@interface NSString (A)  
- (NSString *)tagline;  
@end  
@implementation NSString (A)  
- (NSString *)tagline { return @"из A"; }  
@end  
  
@interface NSString (B)  
- (NSString *)tagline;  
@end  
@implementation NSString (B)  
- (NSString *)tagline { return @"из B"; }  
@end
```

Подумай сам. Гарантирует ли язык, какая реализация окажется в таблице методов?

Разбор. Это **неопределённое поведение**, и так на собеседовании и отвечают. Когда две категории одного класса определяют метод с одним именем, какая из них «ляжет» в таблицу методов класса последней — **не специфицировано**. Обычно побеждает категория, чей объектный файл runtime загрузил позже (зависит от порядка линковки), но опираться на это нельзя.

Хуже того — категория, переопределяющая метод **самого класса** (не другой категории, а, скажем, родной метод NSString), молча затирает оригинал, и вызвать прежнюю реализацию через `super` уже не получится. Поэтому правила такие:

- **Не переопределяй существующие методы через категорию.** Категория — чтобы **добавлять** методы, а не подменять. Хочешь изменить поведение — делай подкласс.
- Имена методов в категориях **префиксируй** (`xyz_tagline`), чтобы не столкнуться ни с будущими методами Apple, ни с чужой категорией. Особенно это важно для категорий на классах Foundation.

Запускать такой код как «эксперимент» бессмысленно: результат у тебя может оказаться из А, у коллеги — из В, а после обновления компилятора — снова другой. Правильный ответ — «поведение не определено, так делать нельзя».

Задача 11. Категория не может добавить `ivar` — как обойти

Условие. Тебе нужно «прицепить» дополнительное поле к чужому классу (скажем, `NSString`) через категорию. Но категория не умеет добавлять `ivar`. Как сохранить данные на объекте?

Подумай сам. Где хранить значение, если места под `ivar` в объекте нет?

Разбор. Категория добавляет **методы**, но не **переменные экземпляра**: размер объекта фиксируется при компиляции класса, и категория его менять не может. Объявить в категории `@property` можно, но автоматического `ivar` под него не появится — будут только объявления геттера и сеттера, которые ты обязан написать сам. А хранить значение — через **связанные объекты** (associated objects) из runtime:

```
#import <objc/runtime.h>
static const char kTagKey; // адрес переменной = уникальный ключ

- (void)setTag:(NSString *)tag {
    objc_setAssociatedObject(self, &kTagKey, tag,
                            OBJC_ASSOCIATION_COPY_NONATOMIC);
}
- (NSString *)tag {
    return objc_getAssociatedObject(self, &kTagKey);
}
```

`objc_setAssociatedObject` привязывает значение к объекту по **ключу** — им служит адрес статической переменной (`&kTagKey`), он гарантированно уникален. Политика `OBJC_ASSOCIATION_COPY_NONATOMIC` означает «храни копию, без атомарности» — это аналог атрибута `copy` у свойства. Runtime сам снимет связанные объекты, когда носитель умрёт, — следить за их освобождением не нужно.

Это ровно тот механизм, на котором, например, держат «обёрнутый» делегат или флаг на ячейке таблицы, не плодя подкласс.

Решение: code/middle-07-associated.m.

```
до: tag = (null)
после: tag = важный
```

До присваивания связанного значения нет — геттер возвращает nil. После — строка достаётся обратно с того же объекта.

Задача 12. KVC: доступ по строке, keyPath и операторы

Условие. Что напечатает этот код? Объясни valueForKey:, valueForKeyPath: и операторы @sum/@avg/@max.

```
[e setValue:@350000 forKey:@"salary"];
[e valueForKeyPath:@"address.city"];
[team valueForKeyPath:@"@sum.salary"];
[team valueForKeyPath:@"@avg.salary"];
```

Подумай сам. Как KVC превращает строку @"salary" в вызов сеттера, и что делает @ перед именем оператора?

Разбор. KVC (Key-Value Coding, глава 20) даёт доступ к свойствам по **строковому имени**, известному только во время выполнения:

- setValue:forKey: ищет сеттер setSalary: и зовёт его. Свойство — примитив NSInteger, поэтому KVC сам **распаковывает** переданный NSNumber (@350000) в NSInteger. Обратно, valueForKey: для примитива сам **упаковывает** значение в NSNumber.
- valueForKeyPath:@"address.city" идёт по точке сквозь вложенный объект: сначала address, у него — city. Это **keyPath**, цепочка ключей.
- Применённый к **массиву**, valueForKey:@"name" собирает значения ключа из всех элементов в новый массив.
- @sum, @avg, @max, @count — **коллекционные операторы**. @ перед именем оператора отличает его от обычного ключа. @"@sum.salary" означает «просуммируй salary по всем элементам массива». Результат — NSNumber.

Решение: code/middle-09-kvc.m.

```
valueForKey name = Acet
после setValue salary = 350000
```

```
keyPath city = Алматы
все имена: (
    "\U0410\U0441\U0435\U0442",
    "\U0411\U043e\U043b\U0430\U0442",
    "\U0414\U0430\U043d\U0430"
)
@sum.salary = 1200000
@avg.salary = 400000
@max.salary = 500000
@count      = 3
```

Кириллица в массиве показана escape-кодами вида \U0410 — так NSArray печатает строки в description, это нормально. Сумма трёх зарплат — 1 200 000, среднее — 400 000, максимум — 500 000.

Задача 13. KVO: что приходит в observeValueForKeyPath: и почему ivar молчит

Условие. Подписались на свойство balance через KVO. Меняем его то через сеттер, то напрямую через ivar. Когда придёт уведомление, а когда нет? Что лежит в словаре change? Что обязательно сделать в конце?

Подумай сам. На чём именно стоит KVO — на присваивании ivar или на вызове сеттера?

Разбор. KVO (Key-Value Observing, глава 20) уведомляет наблюдателя об изменении свойства. Подписка — addObserver:forKeyPath:options:context:. Уведомление приходит в observeValueForKeyPath:ofObject:change:context:, и в словаре change лежат старое и новое значения под ключами NSKeyValueChangeOldKey и NSKeyValueChangeNewKey (если при подписке запросил опции Old и New).

Ключевой момент: **KVO перехватывает сеттер, а не присваивание ivar.** Под капотом runtime подменяет класс объекта на сгенерированный подкласс, чей сеттер setBalance: шлёт уведомления вокруг настоящего присваивания. Поэтому:

- acc.balance = 100 идёт через сеттер — **уведомление придёт.**
- acc->_rawBalance = 999 пишет прямо в ivar, минуя сеттер, — **никакого уведомления.** KVO про такое изменение не знает.

И обязательное: перед смертью наблюдаемого объекта (или наблюдателя) надо снять подписку — removeObserver:forKeyPath:. Забудешь — при следующем уведомлении KVO

попыруется достучаться до уже мёртвого наблюдателя и уронит программу, либо рантайм выдаст «was deallocated while key value observers were still registered».

Решение: code/middle-10-kvo.m.

```
--- через сеттер (KVC-совместимо) ---
изменилось 'balance': 0 -> 100
изменилось 'balance': 100 -> 250
--- прямое изменение ivar (в обход сеттера) ---
_rawBalance стал 999, но observe не вызвался
наблюдатель снят
```

Два изменения через сеттер дали два уведомления со старым и новым значением. Прямая запись в ivar прошла молча. В конце — removeObserver:.

Задача 14. Реализуй NSCopying

Условие. Сделай свой класс копируемым: реализуй copyWithZone: так, чтобы [obj copy] возвращал **независимую** копию (меняем копию — оригинал не дрогнул).

Подумай сам. Что должен вернуть copyWithZone: и как перенести в копию все поля, включая строковое?

Разбор. Чтобы объект отвечал на copy, класс объявляет протокол <NSCopying> и реализует единственный его метод — copyWithZone:. Сам copy без аргументов — это удобная обёртка из NSObject, которая зовёт copyWithZone:nil. Реализация создаёт новый объект и переносит в него состояние:

```
- (id)copyWithZone:(NSZone *)zone {
    Vec2 *copy = [[[self class] allocWithZone:zone] init];
    copy.x = self.x;
    copy.y = self.y;
    copy.label = self.label;    // свойство copy само снимет снимок строки
    return copy;
}
```

Тонкости, которые проверяют на собеседовании:

- [[self class] allocWithZone:zone], а не [[Vec2 alloc] ...] — чтобы у **подкласса** копия получалась правильного класса.

- Строковое поле `label` объявлено свойством `copy`, поэтому присваивание `copy.label = self.label` само сделает снимок — глубоко копировать строку вручную не нужно.
- Числа `x`, `y` — значения, копируются присваиванием.

Решение: `code/middle-11-nscopying.m`.

```
оригинал: (1, 2) A
копия:    (100, 2) B
разные объекты? да
```

Изменили копии `x` и `label` — оригинал остался `(1, 2) A`. Объекты разные: `copy` вернул новый экземпляр, а не ту же ссылку.

Задача 15. Реализуй `isEqual:` и `hash` вместе

Условие. Реализуй сравнение объектов по значению: два «денежных» объекта равны, если равны их суммы. Что будет, если переопределить `isEqual:`, но **забыть** `hash`? Проверь поведение в `NSSet`.

Подумай сам. Зачем коллекциям-множествам нужен `hash`, если есть `isEqual:`?

Разбор. Контракт `NSObject` жёсткий: **если два объекта равны по `isEqual:`, у них обязан совпадать `hash`**. `NSSet` и `NSDictionary` сначала раскладывают объекты по «корзинам» по их `hash` и только внутри корзины сравнивают через `isEqual:`. Если переопределить `isEqual:`, но не `hash`, два «равных» объекта унаследуют дефолтный `hash` от `NSObject` (он основан на адресе) — у каждого свой. Они попадут в **разные корзины**, и множество их сравнивать не станет: оба окажутся внутри, а поиск по «равному» объекту провалится.

Поэтому переопределяют **обе** вещи согласованно:

```
- (BOOL)isEqual:(id)other {
    if (self == other) return YES;
    if (![other isKindOfClass:[Money class]]) return NO;
    return self.amount == ((Money *)other).amount;
}
- (NSUInteger)hash { return (NSUInteger)self.amount; }
```

`hash` должен зависеть **ровно от тех полей**, что участвуют в `isEqual:`. Для нескольких полей их хэши обычно комбинируют через XOR ($a \oplus b$).

Решение: `code/middle-12-isequal-hash.m`.

```
=== сломанный (hash не согласован) ===  
в множестве объектов: 2 (ждали 1)  
containsObject(100)? НЕТ  
=== правильный (isEqual + hash) ===  
в множестве объектов: 1 (ждали 1)  
containsObject(100)? да
```

Без согласованного hash множество приняло два «равных» объекта как разные (count = 2) и не нашло объект по значению. С правильным hash — дубликат отброшен (count = 1), поиск работает.

Задача 16. isEqual: против isEqualToString:

Условие. В чём разница между [a isEqual:b] и [a isEqualToString:b] для двух NSString? Что быстрее и что безопаснее?

Подумай сам. Какой из методов знает заранее, что оба аргумента — строки?

Разбор. Оба сравнивают строки по содержимому (не по адресу — для адреса был бы ==, и для строк он почти всегда неверен). Разница в типе и цене:

- isEqual: — общий метод NSObject. Он принимает id (любой объект), поэтому сначала **проверяет класс** аргумента: а строка ли это вообще? Только потом сравнивает содержимое. Универсален, чуть медленнее, безопасен к чужим типам — [str isEqual:someArray] спокойно вернёт NO.
- isEqualToString: — специализированный метод NSString. Он уже **предполагает**, что аргумент — строка, пропускает проверку класса и сравнивает содержимое напрямую. Быстрее на горячем пути.

Ловушка isEqualToString:: если передать в него **не строку** (например, nil пройдёт молча и вернёт NO, а вот объект другого класса в некоторых случаях — неопределённое поведение), гарантий меньше. Практическое правило: когда **точно знаешь**, что оба — NSString, бери isEqualToString: (читаемее и быстрее). Когда тип под вопросом или сравниваешь разнородные объекты — isEqual:. И никогда не сравнивай строки через ==: он сверяет указатели, а не текст, и «совпадёт» лишь случайно (например, на интернированных литералах).

Задача 17. Быстрый разбор: сработает ли dealloc?

Условие. Для каждого мини-сценария скажи, сработает ли dealloc объекта А к концу блока. Все свойства — под ARC.

1. А создан локально, в конце блока локальная ссылка выходит из области.
2. А сильно держит В, В сильно держит А.
3. А сильно держит В, В держит А через weak-свойство.
4. А хранит в свойстве сору-блок, блок упоминает self.name.
5. А хранит блок, блок упоминает weakSelf.name.

Подумай сам. В каждом случае посчитай, остаётся ли на А хоть одна сильная ссылка после выхода из блока.

Разбор.

1. **Сработает.** Единственная сильная ссылка вышла из области — счётчик 0, dealloc.
2. **Не сработает.** Классический цикл: каждый держит другого сильно, счётчики застряли на 1. Утечка.
3. **Сработает у обоих.** weak не считается за владение: когда исчезли локальные ссылки, А обнулится, за ним В.
4. **Не сработает.** Блок захватил self сильно, self держит блок — цикл «объект → блок».
5. **Сработает.** Блок захватил weakSelf, сильного кольца нет.

Это сжатый обзор всего, что было выше: единственный вопрос на собеседовании по памяти — «остаётся ли на объект сильная ссылка». Если да и она по кругу ведёт к самому объекту — это цикл, dealloc молчит, и одну ссылку в кольце надо сделать weak.

Задача 18. Code review: найди и почини утечку в блоке

Условие. Тебе прислали на ревью этот метод. Объект Timer устанавливает повторяющийся обработчик, который пишет в свой log. Найди проблему и предложи правку.

```
@interface Timer : NSObject
@property (nonatomic, copy) void (^onTick)(void);
@property (nonatomic, strong) NSMutableArray *log;
@end

@implementation Timer
- (void)start {
    self.onTick = ^{
```

```
        [self.log addObject:@"tick"];    // ← что не так?
    };
}
- (void)dealloc { NSLog(@"dealloc Timer"); }
@end
```

Подумай сам. Что захватывает блок в строке `[self.log addObject:...]`?

Разбор. Блок упоминает `self.log` — а значит захватывает **self сильно**. Свойство `onTick` объявлено `copy` и сильно держит блок. Кольцо `self → onTick → self` замкнулось: `Timer` никогда не вызовет `dealloc`, сообщение «`dealloc Timer`» не появится. Это утечка, и компилятор честно предупредит про `-Warc-retain-cycles`.

Правка — слабая копия `self`:

```
- (void)start {
    __weak typeof(self) weakSelf = self;
    self.onTick = ^{
        [weakSelf.log addObject:@"tick"];
    };
}
```

Теперь блок захватывает `weakSelf`, сильного кольца нет, и когда внешний код отпустит `Timer`, тот спокойно уничтожится — `dealloc Timer` напечатается. Если внутри блока к `self` обращаются несколько раз и важно, чтобы он не исчез посреди работы блока, в начале блока берут сильную копию на время вызова:

```
self.onTick = ^{
    __strong typeof(weakSelf) strongSelf = weakSelf;
    if (strongSelf == nil) { return; }    // объект уже умер — выходим
    [strongSelf.log addObject:@"tick"];
};
```

Это «weak-strong dance»: слабая ссылка рвёт цикл, а локальная сильная на время выполнения блока гарантирует, что объект не исчезнет между двумя строками. На собеседовании достаточно назвать причину (захват `self`) и базовую правку (`weakSelf`); упоминание `strongSelf` — плюс к ответу.

Что проверяет этот раздел

Middle-уровень — это уже не «знаю синтаксис», а «понимаю модель». Сведём ключевое в одну картину:

- **Память.** Сильная ссылка владеет, слабая наблюдает и сама становится `nil`, `unsafe_unretained` повисает. Цикл удержания — два владельца по кругу; лечится одним `weak`. ARC циклы не ловит — это на тебе.
- **Блоки.** Захват — копия значения в момент создания; `__block` — общая ячейка; упоминание `self` в блоке свойства — цикл, режется `weakSelf`.
- **Копирование.** `copy/mutableCopy` поверхностны; строковые свойства — `copy`; свой класс делаешь копируемым через `NSCopying`.
- **Контракты.** Равенство — это `isEqual:` плюс согласованный `hash`; словарь копирует ключи; делегат — `weak` и `respondsToSelector:` для `@optional`.
- **Runtime-приёмы.** Категория добавляет методы, но не `ivar` (выручают связанные объекты); две категории с одним методом — неопределённость; KVC даёт доступ по строке, KVO стоит на сеттере и требует `removeObserver:`.

Если на собеседовании ты на каждый такой вопрос отвечаешь не «по памяти», а выводом из модели владения и контрактов классов — это и есть Middle.

Документация Apple

- Programming with Objective-C → **Working with Blocks** — захват переменных, `__block`, циклы удержания с `self` — developer.apple.com/library.
- **Transitioning to ARC Release Notes** — `strong/weak/unsafe_unretained`, захват `self` в блоках, разрыв циклов — developer.apple.com/library.
- **Advanced Memory Management Programming Guide** — модель владения, `autorelease`, пулы автоосвобождения — developer.apple.com.
- **Key-Value Coding Programming Guide** — `valueForKey:`, `valueForKeyPath:`, коллекционные операторы `@sum/@avg/@max` — developer.apple.com.
- **Key-Value Observing Programming Guide** — подписка, словарь `change`, ручные и автоматические уведомления — developer.apple.com.
- `NSCopying` и `-[NSObject copy]` — developer.apple.com/documentation/foundation/nscopying
- `-[NSObject isEqual:]` и `-[NSObject hash]` (контракт равенства) — developer.apple.com/documentation/object
- `-[NSString isEqualToString:]` — developer.apple.com/documentation/foundation/nsstring
- **Associated Objects** — `objc_setAssociatedObject` / `objc_getAssociatedObject` — developer.apple.com/documentation/objectivec
- `NSCache` — developer.apple.com/documentation/foundation/nscache

Приложение F. Задачи собеседования: Senior

Junior проверял механику, Middle — модель владения и контракты классов. На Senior спрашивают про то, что крутится **под капотом**: как устроены isa и метаклассы, как objc_msgSend ищет метод, три шанса пересылки, swizzling, связанные объекты, KVO изнутри, и отдельно — многопоточность: гонки, синглтоны, барьеры, дедлоки, пулы автоосвобождения в фоне. Здесь почти нет ответов в одну строку: за каждым вопросом стоит runtime или модель потоков.

Формат прежний. Сначала **условие**. Если задача про «что выведет» — остановись на **подумай сам** и ответь, прежде чем читать **разбор**. Где есть рабочий код, он лежит в code/senior-NN-имя.m, собран и запущен, и весь показанный вывод — настоящий. Дату, имя программы и число в скобках, которые NSLog печатает в начале строки, здесь убрали — у тебя они будут свои.

Собрать и запустить любой файл:

```
clang -fobjc-arc -framework Foundation -Wall -Wextra -O2 \
    code/senior-01-metaclass.m -o /tmp/t && /tmp/t
```

Часть задач — концептуальные, без файла в code/: они или повесили бы проверку (дедлок), или их результат не определён (две категории), или это разбор словами (исключения, пулы в фоне). Такие помечены прямо в тексте.

Идём от runtime к потокам и краевым случаям, от сложного к очень сложному.

Задача 1. Куда указывает isa класса и где живут методы класса

Условие. Есть класс Tool с методом экземпляра -use и методом класса +build. Ответь, не запуская:

- куда указывает isa самого класса Tool;

- в чьей таблице методов лежит `-use`, а в чьей `+build`;
- куда в итоге приходит цепочка `isa`, если идти объект $\rightarrow$ класс $\rightarrow$ метакласс $\rightarrow$

Подумай сам. Если объект знает свой класс через `isa`, то что знает через свой `isa` сам класс?

Разбор. Объект — это структура, чьё первое поле `isa` указывает на класс (глава 14). Но **класс — тоже объект**, и у него тоже есть `isa`. Указывает он на **метакласс** (`metaclass`) — «класс класса».

Зачем метакласс вообще нужен. Методы экземпляра (с минусом, `-use`) лежат в таблице методов **класса**: шлешь сообщение объекту — runtime идёт по его `isa` в класс и ищет там. А методы класса (с плюсом, `+build`) надо где-то держать по той же схеме. Их кладут в **метакласс**. Когда шлешь сообщение классу (`[Tool build]`), runtime идёт по `isa` класса в метакласс и ищет `+build` там. Одна и та же машина поиска работает на двух уровнях.

Поэтому:

- `-use` лежит в таблице **класса** `Tool`, в метаклассе его нет;
- `+build` лежит в таблице **метакласса** `Tool`, в самом классе его нет.

Цепочка `isa` из примера: объект `t` $\rightarrow$ класс `Tool` $\rightarrow$ метакласс `Tool` $\rightarrow$ дальше метаклассы всех родителей сходятся к **корневому метаклассу** (метакласс `NSObject`), а его `isa` указывает сам на себя — цепочка замыкается. В нашем выводе видно первый шаг этого пути: `isa` метакласса `Tool` ведёт к `NSObject` (вверх по метаклассовой ветке).

Решение — `code/senior-01-metaclass.m`:

```
isa класса Tool указывает на: Tool (метакласс? да)
-use   в классе Tool?      да
-use   в метаклассе Tool?  нет
+build в классе Tool?      нет
+build в метаклассе Tool?  да
цепочка isa:
  объект t      isa -> Tool
  класс Tool    isa -> Tool
  метакласс Tool isa -> NSObject
```

Заметь тонкость вывода: у метакласса **то же имя**, что у класса, — `Tool`. `class_getName` для метакласса возвращает имя класса. Отличить их можно только через `class_isMetaClass` (в первой строке — «метакласс? да»). А `-use` нашёлся только в классе, `+build` — только в метаклассе: ровно там, где runtime их и держит.

Отличие от Си. В Си у типа нет «класса своего типа» — после компиляции от `struct Tool` не остаётся ничего, спрашивать не у кого. Здесь же у класса есть свой класс

(метакласс), а у того — свой (корневой метакласс): целая живая лесенка `isa` там, где в Си пустота.

Задача 2. `objc_msgSend`, `SEL`, `IMP`, `Method` — кто есть кто

Условие. У класса `Calc` есть метод `-(int)addTo:(int)x`, который возвращает `x + 100`. Объясни, во что компилятор превращает `[c addTo:5]`, и что напечатают три вызова — обычная отправка, вызов через `IMP` напрямую и вызов через `objc_msgSend`, приведённый к точной сигнатуре.

```
int viaMessage = [c addTo:5];           // (1)
int (*impFn)(id, SEL, int) = (void *)impFromMethod;
int viaImp = impFn(c, sel, 5);          // (2)
int (*send)(id, SEL, int) = (void *)objc_msgSend;
int viaSend = send(c, sel, 5);          // (3)
```

Подумай сам. Чем `SEL` отличается от `IMP`, а `Method` — от них обоих?

Разбор. Разложи отставку сообщения на детали (глава 14):

- **SEL (селектор)** — уникализированное имя сообщения. Не строка, а её зарегистрированный в runtime «номерок»: для `"addTo:"` во всей программе существует ровно один `SEL`, поэтому селекторы сравнивают через `==`.
- **IMP (implementation)** — указатель на си-функцию, которая и есть тело метода. Её первые два скрытых аргумента — `self` и `_cmd` (селектор), дальше параметры.
- **Method** — пара (`SEL`, `IMP`) плюс кодировка типов. Класс хранит таблицу таких пар.
- **`objc_msgSend(receiver, sel, ...)`** — функция, которая по `isa` получателя находит нужную `IMP` и передаёт ей управление.

`[c addTo:5]` компилятор превращает в `objc_msgSend(c, @selector(addTo:), 5)`. Все три пути в задаче ведут к одной и той же `IMP` и дают один и тот же результат — `105`. Разница лишь в том, **на каком уровне** тыходишь: скобки прячут `objc_msgSend`; `impFn` зовёт реализацию напрямую, минуя поиск; `send` — это сам `objc_msgSend`, приведённый к сигнатуре метода (под `-Wall` без приведения её звать нельзя).

Кодировка типов `i20@0:8i16` читается так: возвращает `int (i)`; дальше аргументы — `@(self), :(_cmd), i` (наш `int x`); числа — смещения и размеры, их можно игнорировать. Аргументов «3» — это `self`, `_cmd` и `x`: два всегда скрытые.

Решение — `code/senior-02-msgsend.m`:

```

SEL как строка: addTo:
Method найден: да
кодировка типов метода: i20@0:8i16
число аргументов (с учётом self и _cmd): 3
IMP из Method и из класса совпали? да
[c addTo:5]          = 105
через IMP            = 105
через objc_msgSend = 105

```

IMP из Method (через `method_getImplementation`) и IMP из класса (через `class_getMethodImplementation`) совпали — это буквально один адрес кода. Все три вызова дали 105: квадратные скобки — лишь удобный синтаксис над `objc_msgSend`.

Отличие от Си. В Си `addTo(c, 5)` — переход по **фиксированному адресу**, выбранному компоновщиком на этапе сборки. В Objective-C тот же вызов — это поиск IMP по SEL **во время выполнения**, от `isa` получателя вверх. Один и тот же `[obj addTo:5]` может выполнить разный код в зависимости от того, какой объект сейчас в `obj`.

Задача 3. Динамическое разрешение метода: сколько раз позовётся `resolve`

Условие. Класс `Ghost` не реализует ни `shout`, ни `twice`. Их выдаёт `+resolveInstanceMethod:` через `class_addMethod` при первом обращении. Сколько раз вызовется `+resolveInstanceMethod:`, если послать `shout` дважды и `twice`: один раз?

Подумай сам. Когда метод уже доращён в класс, нужен ли `resolve` при следующем таком же сообщении?

Разбор. Когда `objc_msgSend` не находит метод обычным поиском (по классу и вверх по `superclass`), он даёт классу первый шанс — зовёт `+resolveInstanceMethod:(SEL)sel:` «может, доопределишь?» Если ты внутри вызвал `class_addMethod` и вернул YES, runtime повторяет поиск — теперь метод на месте.

Ключевое: `class_addMethod` встраивает метод **в класс навсегда**. Поэтому `resolve` срабатывает **ровно один раз на селектор** — при самом первом сообщении. Второй `shout` идёт уже по готовой таблице, минуя разрешение.

Считаем вызовы `+resolveInstanceMethod:`:

- первый shout → метода нет → resolve (раз), доращиваем, выполняем;
- второй shout → метод уже есть → resolve **не** зовётся;
- twice: → метода нет → resolve (два), доращиваем, выполняем.

Итого — **2 раза**. Кодировки типов в `class_addMethod: "@@:"` — вернёт объект (@), принимает `self` (@) и `_cmd` (:); "@@:@" — то же плюс ещё один объектный аргумент.

Решение — `code/senior-03-resolve.m`:

```
первый вызов shout:
[resolve] доопределили shout
Ghost кричит! (селектор был shout)
второй вызов shout (resolve уже не нужен):
Ghost кричит! (селектор был shout)
вызов twice:21 ->
[resolve] доопределили twice:
42
resolveInstanceMethod: позвался 2 раз(a)
```

[resolve] напечаталось дважды — по разу на каждый новый селектор. Между двумя shout его нет: класс буквально вырос на метод и больше в разрешении не нуждается.

Задача 4. Логирующий прокси на `forwardingTargetForSelector:`

Условие. Сделай прокси, который сам не реализует `name` и `compute:`, а пересылает их «настоящему» объекту `Service` и попутно считает каждую пересылку. Какой из трёх шансов пересылки тут самый дешёвый и почему?

Подумай сам. Что должен вернуть `forwardingTargetForSelector:`, чтобы runtime просто повторил сообщение другому объекту?

Разбор. Когда первый шанс (`resolveInstanceMethod:`) не сработал, runtime даёт **второй**: `-forwardingTargetForSelector:(SEL)sel`. Ты возвращаешь **другой объект**, и runtime повторяет отправку уже ему — буквально как `objc_msgSend(другойОбъект, sel, ...)`. Это самый дешёвый шанс: ни копирования аргументов, ни построения `NSInvocation`, сообщение просто едет к новому получателю. Тип возврата и аргументы сохраняются бесплатно.

На этом строят прокси: объект-обёртка хранит `_target` и в `forwardingTargetForSelector:` возвращает его, если тот умеет селектор. Заодно можно посчитать и залогировать каждую пересылку — то, ради чего обёртку часто и заводят (аналитика, трассировка).

Тонкость: `[proxy respondsToSelector:@selector(name)]` вернул бы **нет** — прокси и правда не реализует `name`, он его пересылает. Пересылка работает уровнем ниже, уже после того как обычный поиск метода провалился.

Решение — `code/senior-05-logging-proxy.m`:

```
[proxy] переслано: name (всего 1)
name      = Service
[proxy] переслано: compute: (всего 2)
compute = 81
[proxy] переслано: compute: (всего 3)
compute = 16
итого переслано сообщений: 3
```

Снаружи проху ведёт себя как `Service`: вернул имя и посчитал квадраты ($9^2 = 81$, $4^2 = 16$). Внутри — три залогированные пересылки. Один объект прозрачно подменил другой, добавив к нему лог.

Задача 5. Рассылающий прокси: почему тут нужен `forwardInvocation`:

Условие. Нужен прокси, который **одно** сообщение `event:` доставляет **сразу нескольким** адресатам. Почему `forwardingTargetForSelector:` тут не годится и какой шанс пересылки придётся взять?

Подумай сам. Сколько объектов умеет вернуть `forwardingTargetForSelector:`?

Разбор. `forwardingTargetForSelector:` умеет вернуть **ровно одного** получателя — `runtime` повторит сообщение ему, и на этом всё. Доставить одно сообщение нескольким он не может в принципе.

Здесь нужен **третий шанс** — самый мощный и самый дорогой:

1. `-methodSignatureForSelector:` — `runtime` просит описание сигнатуры метода, чтобы понять, как упаковать аргументы. Берём сигнатуру у первого адресата, который знает селектор.
2. `-forwardInvocation:(NSInvocation *)inv` — `runtime` упаковывает всё сообщение целиком в объект `NSInvocation`: получатель, селектор, все аргументы. С ним можно делать что угодно. Мы в цикле зовём `[inv invokeWithTarget:t]` для **каждого** адресата — сообщение выполняется на всех по очереди.

NSNotification строить дороже, чем просто повторить отправку, но только он даёт полный контроль: переслать многим, изменить аргументы, отложить, залогировать. Тип переменной — `id<Listener>`: компилятор разрешает слать `event:`, хотя `Broadcaster` его не реализует (динамика главы 5).

Решение — `code/senior-04-broadcast-proxy.m`:

шлём ОДНО сообщение прокси:

```
Анна услышал событие: пожарная тревога
Борис услышал событие: пожарная тревога
Вера услышал событие: пожарная тревога
```

Одно `[group event:@"пожарная тревога"]` превратилось в три доставки. `forwardingTargetForSelector:` так не смог бы — он вернул бы лишь одного слушателя.

Задача 6. Ленивый прокси на `NSProxy`: зачем отдельный корень

Условие. «Тяжёлый» объект `Heavy` дорого создавать, и мы хотим отложить его создание до первого реального обращения. Сделай прокси-заместитель. Почему для этого берут `NSProxy`, а не `NSObject`?

Подумай сам. Если унаследовать прокси от `NSObject`, какие сообщения он обработает **сам**, не переслав их `Heavy`?

Разбор. В Objective-C два корня иерархии (глава 14). Основной — `NSObject`: он тащит гору готового поведения (`description`, `hash`, `isEqual:`, `respondsToSelector:`, десятки `NSObject`-методов). Второй, отдельный — `NSProxy`: тонкий абстрактный класс специально для заместителей. Он почти ничего не реализует сам и заставляет наследника определить пересылку (`methodSignatureForSelector:` + `forwardInvocation:`).

В этом и причина выбора. Прокси должен пересылать `Heavy` **всё**. Унаследуй он `NSObject` — десятки сообщений (`description`, `isEqual:`, ...) прокси обработал бы **сам**, своим унаследованным кодом, и они никогда не дошли бы до `Heavy`. У `NSProxy` такого «лишнего» поведения почти нет: мимо пересылки проходит минимум, и `Heavy` получает практически все сообщения.

Механика: `LazyProxy` хранит `_real` (поначалу `nil`). Метод `real` создаёт `Heavy` при первом обращении (лениво). `forwardInvocation:` зовёт `[inv invokeWithTarget:[self real]]` — и именно тут, на первом реальном сообщении, `Heavy` наконец рождается. Создаётся прокси через `[LazyProxy alloc]` без `initWith:` у `NSProxy` своя схема.

Решение — code/senior-13-nsproxy.m:

```
создаём прокси (Heavy ещё НЕ создан):  
прокси готов, Heavy всё ещё не создан  
первое реальное обращение -> report:  
    Heavy: дорогой объект СОЗДАН  
    ответ: тяжёлый отчёт готов
```

Прокси создан, а «Heavy СОЗДАН» появилось **только** на report — создание отложилось ровно до первого дела. Будь прокси на NSObject, часть служебных сообщений он перехватил бы сам, и прозрачной замены не вышло бы.

Задача 7. Method swizzling правильно: +load, dispatch_once, class_addMethod

Условие. Нужно подменить метод `-tap` класса `Button` так, чтобы перед оригиналом срабатывала аналитика. Где безопасно делать подмену, зачем тут `dispatch_once`, и почему «правильный» swizzle сначала пробует `class_addMethod`, а не сразу `method_exchangeImplementations`?

Подумай сам. Что будет, если метод `tap` достался классу **по наследству**, а ты обменяешь его реализацию через `exchange`?

Разбор. Swizzling — обмен реализациями двух методов (глава 14). Делать его надо аккуратно, иначе словишь трудноуловимый баг. Три правила из рабочего решения:

1. **Где.** Подмену кладут в `+load` категории. `+load runtime` зовёт один раз при загрузке образа, ещё до `main`, и потокобезопасно. Это надёжный момент: класс уже есть, но программа ещё не работает.
2. **dispatch_once.** Дополнительный пояс поверх подтяжек: гарантирует, что подмена выполнится строго один раз, даже если код как-то позовут повторно. Двойной swizzle вернул бы метод в исходное состояние — `dispatch_once` это исключает.
3. **Сначала class_addMethod.** Если `tap` достался `Button` **по наследству** (определён в родителе), голый `method_exchangeImplementations` полез бы менять реализацию **в родителе** — а это ломает поведение для всех остальных потомков. Поэтому сперва пробуем `class_addMethod`: она добавит метод **в наш класс** и вернёт YES, если метода тут ещё не было. Тогда обмен делаем уже внутри своего класса. И только если метод определён прямо в `Button` (`class_addMethod` вернула NO) — честный `exchange`.

Тело `analytics_tap` после обмена в конце зовёт `[self analytics_tap]` — на вид рекурсия, но это не она: после обмена имя `analytics_tap` указывает на **старый** код `tap`. Классический приём «обёртки».

Решение — code/senior-06-swizzle.m:

нажимаем кнопку:

```
[analytics] зафиксировали нажатие  
Button: обычное нажатие
```

[b tap] теперь сначала логирует, потом выполняет исходное нажатие — мы обернули чужой метод, не трогая его источник. Подмена глобальна: касается всех экземпляров Button в программе.

Задача 8. Найди тонкий баг: swizzling, который ломает KVO

(Концептуальная, без файла — разбираем словами.)

Условие. Команда засвищила сеттер setName: у класса Person (добавили вокруг него логирование) прямо в +load. Работало месяцами. Потом в одном экране на тот же Person повесили KVO — и логирование сеттера то срабатывает, то нет, а иногда наблюдатель вообще не получает уведомлений. Где тонкий баг?

Подумай сам. Что делает KVO с объектом в момент подписки — и как это сочетается с тем, что ты уже подменил setName: на уровне класса?

Разбор. Вспомни, как устроен KVO (глава 14, задача 10 ниже). В момент addObserver: runtime создаёт **динамический подкласс** наблюдаемого объекта (NSKVONotifying_Person), переопределяет в нём setName: так, чтобы вокруг настоящего присваивания слались уведомления, и **подменяет isa** объекта на этот подкласс. То есть KVO ставит свой сеттер **поверх**, на уровне сгенерированного подкласса.

Теперь столкни это со swizzling. Ты обменял реализации setName: на уровне **Person**. Дальше порядок решает всё:

- Объект **без** KVO зовёт setName: → isa ведёт в Person → срабатывает твой засвищенный код. Лог есть.
- Объект **под** KVO зовёт setName: → isa ведёт в NSKVONotifying_Person → срабатывает KVO-сеттер. Он внутри вызывает super-версию (то есть setName: у Person) — и попадает на твой swizzle. Вроде бы и лог, и уведомление. Но это работает **только если** KVO подписался **после** твоей подмены и видит уже подменённую реализацию как «оригинал».

Где рвётся. KVO определяет, нужно ли вообще генерировать подкласс и переопределять сеттер, по тому, что он видит у класса в момент подписки. Если ты свищишь **динамически и поздно** (не в +load, а где-то по ходу программы — особенно уже **после** того, как KVO успел сделать isa-swizzling на каком-то экземпляре), ты меняешь setName: в Person **под ногами**

у KVO. KVO-сеттер в подклассе по-прежнему зовёт `super`, но «оригинал», на который он рассчитывал при подписке, уже не тот — твой обмен переставил IMP местами. Результат: уведомления и логи перестают совпадать, ломается то одно, то другое, в зависимости от того, кто на каком экземпляре успел первым — KVO подменить `isa` или ты подменить метод.

Признаки тонкого бага именно в этом:

- **Порядок `isa`-подмены и `method`-подмены не определён**, если `swizzle` не заперт в `+load`. Динамический `swizzle` сеттера, который наблюдают через KVO, — почти гарантированный источник «плавающих» багов.
- KVO маскирует подмену: `-class` объекта под наблюдением врёт, что это по-прежнему `Person`, хотя настоящий `isa` — `NSKVONotifying_Person`. Поэтому глазами баг не виден: класс «тот же».

Как чинить. Свиззли **только в `+load` и только один раз** (`dispatch_once`) — тогда подмена случается до любого KVO, и KVO видит уже финальную реализацию как оригинал. И не свиззли сеттеры свойств, которые где-то наблюдают через KVO: KVO сам генерирует для них сеттер, и две подмены на одном методе дерутся за порядок. Хочешь и лог, и наблюдение — наблюдай через KVO, а лог вешай не подменой сеттера, а в самом сеттере или через тот же KVO-наблюдатель.

Короткий ответ на собеседовании: **swizzling сеттера + KVO на том же свойстве = гонка двух `isa`/IMP-подмен; лечится переносом `swizzle` в `+load` (до KVO) и отказом от подмены наблюдаемых сеттеров.**

Задача 9. Связанный объект: хранимое свойство в категории

Условие. `View` — будто бы чужой класс из фреймворка: исходник недоступен, `ivar` в него не добавить. Нужно прицепить к нему счётчик `badgeCount`. Категория хранить поля не умеет. Как сохранить значение **на объекте**, и почему у двух разных `View` оно своё?

Подумай сам. Если в объекте нет места под новый `ivar`, где `runtime` держит привязанное к нему значение?

Разбор. Категория добавляет **методы**, но не `ivar`: размер объекта фиксируется при компиляции класса. Объявить в категории `@property` можно, но автоматического `ivar` под него не будет — только обещание геттера и сеттера, которые ты обязан написать сам. А значение хранят через **связанные объекты** (`associated objects`) из `runtime`:

```
static const void *kBadgeKey = &kBadgeKey;    // адрес = уникальный ключ
```

```

- (void)setBadgeCount:(NSNumber *)v {
    objc_setAssociatedObject(self, kBadgeKey, v,
                             OBJC_ASSOCIATION_RETAIN_NONATOMIC);
}
- (NSNumber *)badgeCount {
    return objc_getAssociatedObject(self, kBadgeKey);
}

```

objc_setAssociatedObject привязывает значение к **конкретному объекту** по ключу. Ключом служит адрес статической переменной (&kBadgeKey) — он гарантированно уникален во всей программе. Привязка живёт у каждого экземпляра своя: runtime держит её в отдельной таблице, индексированной по объекту, а не в самом объекте. Поэтому a.badgeCount и b.badgeCount независимы. И следить за освобождением не нужно: связанные объекты runtime снимает сам, когда носитель умирает.

Политика OBJC_ASSOCIATION_RETAIN_NONATOMIC означает «держи сильно, без атомарности» — аналог strong, nonatomic у свойства (для строк взяли бы COPY_NONATOMIC).

Решение — code/senior-07-associated.m:

```

до: Почта badge=(null), Чаты badge=(null)
после: Почта badge=7, Чаты badge=0
у каждого объекта своё значение? да

```

До присваивания связанного значения нет — геттер возвращает nil ((null)). После — у «Почты» 7, у «Чатов» 0: значение у каждого объекта своё, хотя iVar мы не добавляли и класс не трогали.

Отличие от Си. В Си, чтобы прицепить данные к чужой структуре, пришлось бы менять её определение и пересобирать всех, кто её использует. Здесь runtime хранит привязку в стороне, по адресу объекта, — чужой класс можно «дорастить» данными, не имея его источника.

Задача 10. KVO под капотом: подмена isa и NSKVONotifying_

Условие. Подписались через KVO на свойство balance объекта Account. Что произойдёт с **классом** объекта? Что покажет -class, а что — реальный isa (через object_getClass) до подписки, после неё и после отписки?

Подумай сам. Как одно лишь `addObserver:` заставляет обычный сеттер рассылать уведомления, не трогая исходный код `Account`?

Разбор. KVO стоит **на сеттере**, и реализован он через подмену `isa` (глава 14). В момент `addObserver: forKeyPath: runtime:`

1. генерирует **динамический подкласс** `NSKVONotifying_Account`;
2. переопределяет в нём `setBalance:` так, чтобы вокруг настоящего присваивания слались `willChangeValueForKey: / didChangeValueForKey:`;
3. **подменяет `isa`** объекта асс на этот новый подкласс.

С этого момента асс физически — экземпляр `NSKVONotifying_Account`, и его сеттер уведомляет наблюдателей. Но `-class` специально **врёт**, что это по-прежнему `Account`: KVO маскирует подмену, чтобы не пугать твой код. А настоящий `isa`, прочитанный напрямую через `object_getClass`, выдаёт правду — `NSKVONotifying_Account`. После `removeObserver: runtime` возвращает `isa` на место — снова `Account`.

Отсюда два важных факта (оба видны в выводе):

- Уведомление приходит только при изменении **через сеттер** — потому что перехвачен именно сеттер. Прямая запись в `ivar` минует его, и KVO молчит.
- В конце обязателен `removeObserver:` — иначе при следующем уведомлении KVO достучится до мёртвого наблюдателя и уронит программу.

Решение — `code/senior-08-kvo-isa.m`:

```
до подписки:  -class=Account  настоящий isa=Account
после подписки: -class=Account  настоящий isa=NSKVONotifying_Account
меняем баланс через сеттер:
[наблюдатель] balance: 0 -> 100
[наблюдатель] balance: 100 -> 250
после отписки:  настоящий isa=Account
```

Вот доказательство «isa-swizzling»: `-class` всё время говорит `Account`, а настоящий `isa` после подписки стал `NSKVONotifying_Account` и вернулся обратно после отписки. Два изменения через сеттер дали два уведомления со старым и новым значением. Это та же машинерия, что в задаче 8, — почему swizzling наблюдаемых сеттеров и опасен.

Задача 11. Две категории с одинаковым методом — какая победит

(Концептуальная, без файла — результат не определён, запускать бессмысленно.)

Условие. На один класс навесили две категории, и в обеих объявлен метод `tagline` с **разной** реализацией. Какая из них вызовется?

```
@implementation NSString (A)
- (NSString *)tagline { return @"из A"; }
@end
@implementation NSString (B)
- (NSString *)tagline { return @"из B"; }
@end
```

Подумай сам. Гарантирует ли язык, какая реализация ляжет в таблицу методов класса?

Разбор. Это **неопределённое поведение**. Когда две категории одного класса определяют метод с одним именем, какая из них «победит» — **не специфицировано**.

Почему так, на уровне runtime. При загрузке образа runtime берёт методы категорий и **дописывает** их в таблицу методов класса. Методы категории кладутся так, что при поиске находятся **раньше** родных методов класса, — поэтому категория и «перебивает» оригинал. А вот порядок между **двумя категориями** определяется порядком, в котором runtime их подгрузил, то есть порядком линковки объектных файлов. По нашим наблюдениям обычно «побеждает» категория, загруженная **последней** (её метод оказывается в начале списка и находится первым), — но опираться на это нельзя: поменяется порядок файлов в сборке, версия компилятора или тулчейна — и результат сменится. Запускать это как «эксперимент» бессмысленно: у тебя выйдет из A, у коллеги из B.

Хуже того — категория, переопределяющая **родной** метод класса (не другой категории, а сам метод `NSString`), молча затирает оригинал, и вызвать прежнюю реализацию через `super` уже нельзя: у категории нет «родителя» с этим методом.

Правила, которые ждут в ответе:

- **Категория — чтобы добавлять методы, а не подменять.** Хочешь изменить поведение — делай подкласс или swizzling (осознанно, см. задачи 7–8).
 - **Префиксируй имена методов в категориях** (`abc_tagline`), особенно на классах Foundation, — чтобы не столкнуться ни с будущим методом Apple, ни с чужой категорией.
 - Правильный ответ на «какая победит» — **«поведение не определено, так делать нельзя»**, а не угадывание порядка линковки.
-

Задача 12. Гонка данных: `atomic` не делает коллекцию потокобезопасной

Условие. Свойство `value` объявлено `atomic`. Два потока параллельно по 100 000 раз делают `value = value + 1`. Ожидаем 200 000. Что получим — и почему `atomic` тут не спас?

Подумай сам. `atomic` делает неделимым **что** — отдельный `get/set` или всю операцию `value = value + 1`?

Разбор. `atomic` гарантирует, что **каждый отдельный** геттер и сеттер выполнится целиком, без того чтобы другой поток увидел свойство «наполовину записанным». Это всё, что он обещает.

`A value = value + 1` — это **две** раздельные операции:

1. `get` — прочитать текущее `value`;
2. `set` — записать `value + 1`.

Между ними другой поток успевает вклиниться: оба читают, скажем, 1000, оба пишут 1001 — один инкремент потерян. `atomic` сделал неделимыми **шаги**, но не **связку** `read-modify-write`. Это и есть гонка данных (`race condition`): часть инкрементов теряется, итог меньше ожидаемого.

Чинить надо так, чтобы **вся** связка `read-modify-write` была неделимой. В решении это последовательная (`SERIAL`) очередь: `dispatch_sync(guard, ^{ good++; })` — два потока физически не пересекутся внутри, очередь пропускает по одному. Альтернативы — мьютекс (`@synchronized`, `os_unfair_lock`) или атомарный счётчик из `<stdatomic.h>`.

Решение — `code/senior-09-atomic-race.m`:

```
atomic-свойство: ожидали 200000, получили 175216 (часть потеряна)
serial-очередь:  ожидали 200000, получили 200000
```

Точное число у тебя будет своё (гонка недетерминирована), но оно будет **меньше** 200 000 — инкременты теряются. Через последовательную очередь — ровно 200 000.

Вывод, который ждут на собеседовании: **`atomic` защищает доступ к одному свойству, а не последовательность операций и тем более не содержимое коллекции.** `atomic`-свойство типа `NSMutableArray` тоже не делает сам массив потокобезопасным — атомарна лишь подмена указателя на массив, а не `addObject:`. Для составных операций нужна явная синхронизация.

Задача 13. `dispatch_sync` на текущей последовательной очереди — дедлок

(Концептуальная, без файла — повесила бы проверку.)

Условие. Что произойдёт с этим кодом, если он выполняется **на главном потоке**?

```
dispatch_sync(dispatch_get_main_queue(), ^{
    NSLog(@"привет");
});
```

А с этим — если он уже выполняется внутри блока на последовательной очереди q?

```
dispatch_sync(q, ^{                // мы уже внутри блока на q
    dispatch_sync(q, ^{ ... });    // и снова синхронно на ту же q
});
```

Подумай сам. Что делает `dispatch_sync` с **текущим** потоком, пока блок не выполнится, — и кто на последовательной очереди должен этот блок выполнить?

Разбор. Оба случая — **дедлок** (взаимная блокировка), программа зависает навсегда.

Разбери `dispatch_sync` по словам: «поставь блок в очередь и **заблокируй текущий поток**, пока этот блок не выполнится до конца». Теперь наложи это на **последовательную** очередь — она выполняет блоки строго по одному, не начиная следующий, пока не закончит текущий.

Первый случай. Главная очередь последовательна, и обслуживает её главный поток. Ты на главном потоке зовёшь `dispatch_sync(main)`. Главный поток **блокируется** и ждёт, пока блок выполнится. Но выполнить блок может только главный поток — а он сейчас стоит и ждёт сам себя. Никто не сдвинется: зависание.

Второй случай — та же ловушка в общем виде. Ты **внутри** блока на очереди q синхронно ставишь на q ещё один блок и ждёшь его. Но q не начнёт второй блок, пока не доделает первый, — а первый стоит и ждёт второго. Круг замкнулся.

Ключевая формула: **`dispatch_sync` на ту очередь, на которой ты сейчас выполняешься (если она последовательная), — это всегда дедлок.** Сюда же частый прод-баг: код на главном потоке вызывает «удобную» обёртку, а та внутри делает `dispatch_sync(main)` «чтобы наверняка на главном» — и всё виснет, как только её случайно позвали уже с главного потока.

Как правильно:

- Если ты, возможно, **уже** на нужной очереди и нужен синхронный результат — не делай безусловный `dispatch_sync`; проверь, та ли это очередь, или выполни блок напрямую.
- Не нужен результат прямо сейчас — бери `dispatch_async`: он не блокирует текущий поток и дедлока не даёт.
- `dispatch_sync` на **другую** очередь (не текущую) — нормально; опасна именно отправка на самого себя.

В code/ этот пример не кладём намеренно: дедлок повесил бы проверку примеров наглухо.

Задача 14. Потокобезопасный синглтон на `dispatch_once`

Условие. Шестнадцать потоков одновременно лезут за `[Database shared]`. Напиши `+shared` так, чтобы объект создался **ровно один раз** и все потоки получили один и тот же экземпляр. Почему именно `dispatch_once`, а не `if (instance == nil)`?

Подумай сам. Что будет, если два потока одновременно пройдут проверку `if (instance == nil)` раньше, чем хоть один успел присвоить `instance`?

Разбор. Наивный `if (instance == nil) { instance = [[self alloc] init]; }` в многопоточе ломается: два потока могут **одновременно** увидеть `nil`, оба создадут объект, и один экземпляр перезастрёт другой — синглтон перестает быть единственным. Это та же гонка, что в задаче 12, только цена выше: расходятся не числа, а сам объект.

`dispatch_once` решает это на корню. Он гарантирует, что блок выполнится **один раз за всё время работы программы**, даже под шквалом потоков: первый поток выполняет блок, остальные ждут его завершения и затем видят готовый результат. Токен `dispatch_once_t` обязан быть `static` (одна ячейка на всю программу) — по нему `dispatch_once` и помнит, что уже отработал.

Деталь: внутри пишут `[[self alloc] init]`, а не `[[Database alloc] init]`. `self` в методе класса — это сам класс, по которому метод позвали, поэтому подкласс получит синглтон своего класса.

Решение — `code/senior-10-singleton.m`:

```
>>> Database создан (видно один раз)
a == b ? да, один объект
path: /var/db/main
```

Строка «Database создан» напечаталась **один раз**, хотя за синглтоном ломились 16 потоков. `a == b` — один и тот же объект. `dispatch_once` сделал создание атомарным и единственным.

Задача 15. Потокобезопасный кэш: барьерная запись на `concurrent-очереди`

Условие. Нужен кэш «ключ-значение», в который пишут и из которого читают **много потоков сразу**. Хочется, чтобы чтения шли параллельно, а записи — эксклюзивно (никто не читает в момент записи). Как это устроить на GCD?

Подумай сам. Можно ли пускать читателей параллельно, но писателя — в одиночку, на одной и той же очереди?

Разбор. Это классическая схема «много читателей — один писатель» (readers-writer), и GCD даёт её почти даром через **барьер** на собственной **concurrent**-очереди:

- **Чтение** — `dispatch_sync(_queue, ^{ ... })` на `concurrent`-очереди: несколько чтений идут **параллельно**, не мешая друг другу, и `sync` возвращает прочитанное значение вызывающему.
- **Запись** — `dispatch_barrier_async(_queue, ^{ ... })`. Барьер заставляет GCD дожидаться завершения **всех текущих** операций на очереди, выполнить блок-писатель **в одиночку** (в этот момент на очереди больше ничего не идёт) и только потом пустить следующие. `async` — потому что писателю не нужно ждать результата, он лишь ставит барьер в очередь и сразу возвращается, не блокируя свой поток.

Важно: очередь должна быть **своя**, созданная `dispatch_queue_create(..., DISPATCH_QUEUE_CONCURRENT)`. На глобальной `concurrent`-очереди барьер не работает как барьер — она общая, и GCD не даёт на ней эксклюзивности.

Зачем `count` в конце читает через `dispatch_sync` на той же очереди: к моменту, когда `sync`-чтение дойдёт до выполнения, все ранее поставленные барьеры-записи на этой очереди уже отработают (очередь упорядочена) — поэтому увидим ровно 100 элементов, а не «сколько успели».

Решение — `code/senior-11-barrier-cache.m`:

```
в кэше 100 элементов (ожидали 100)
k7  = 49 (ожидали 49)
k12 = 144 (ожидали 144)
```

Сто параллельных писателей и пятьдесят параллельных читателей не подрались: барьер развёл записи с чтениями, и кэш собрал все 100 пар ($7^2 = 49$, $12^2 = 144$). Чтения при этом шли одновременно — барьер сдерживает только запись.

Задача 16. weak в многопоточности и пулы автоосвобождения в фоне

(Концептуальная, без файла — разбираем словами.)

Условие. Две части одного вопроса.

1. Безопасно ли читать __weak-ссылку из нескольких потоков? Что не так с таким кодом, если объект могут освободить в другом потоке?

```
if (self.weakDelegate) {  
    [self.weakDelegate doWork];    // между проверкой и вызовом?  
}
```

2. Ты запускаешь долгую работу на **своём** фоновом потоке (NSThread/ pthread), внутри в цикле создаёшь временные объекты. Почему память может потечь и runtime ругается «autoreleased with no pool in place»?

Подумай сам. (1) Сколько раз этот код **читает** weakDelegate и что может случиться между чтениями? (2) Кто на свеже созданном потоке отвечает за слив пула автоосвобождения?

Разбор, часть 1 — weak и потоки. Само по себе чтение и запись __weak-ссылки **потокобезопасны**: машинерия ARC под капотом синхронизирована, и ты всегда получишь либо валидный, уже удержанный на время чтения объект, либо честный nil, но **никогда** не висячий указатель на освобождённую память. Это сильная гарантия.

Ловушка в другом — в **двойном чтении**. Код выше читает weakDelegate **дважды**: один раз в if, второй раз при вызове. Между ними другой поток может освободить объект. Тогда первое чтение даст живой объект (if пройдёт), а второе — уже nil. Само сообщение к nil безопасно (тихий по-ор), но логика «проверил — значит есть» нарушена, и при более сложном коде это даёт плавающие баги. Правильно — **прочитать один раз в сильную локальную переменную**, она удержит объект на время работы:

```
__strong id strong = self.weakDelegate;    // одно чтение, объект удержан  
if (strong) {  
    [strong doWork];                        // он точно жив до конца блока  
}
```

Это тот же «weak-strong dance», что и в блоках (приложение E): слабая ссылка рвёт цикл, а локальная сильная гарантирует, что объект не исчезнет **посреди** работы.

Разбор, часть 2 — пулы в фоне. Многие объекты Foundation возвращаются **автоосвобождёнными**: их держит в живых только **пул автоосвобождения** (autorelease pool), пока тот не сольётся (глава 13). Вопрос — кто этот пул создаёт.

- На **главном** потоке пул есть всегда — его крутит run loop приложения, сливая на каждом витке.
- На очередях **GCD** система оборачивает выполнение блоков в свой пул и периодически его сливает — поэтому под dispatch_async обычно пул заводить не нужно (но для долгого цикла внутри блока — всё равно стоит).

- На **своём** потоке (NSThread, голый pthread) **пула нет**. Никто его не создал. Все автоосвобождённые объекты, что ты наплодил, повисают без владельца и **текут** — а runtime честно предупреждает в консоль: object ... autoreleased with no pool in place — just leaking.

Лечение — обернуть тело потока (а в длинном цикле — тело итерации) в свой @autoreleasepool:

```
- (void)threadMain {
    @autoreleasepool {                // пул на весь поток
        while (working) {
            @autoreleasepool {        // и на каждую итерацию — пик памяти низкий
                ... временные объекты ...
            }
        }
    }
}
```

Короткий ответ на собеседовании: **weak потокобезопасен, но читай его один раз в сильную локальную; у своего фонового потока нет пула автоосвобождения — оборачивай работу в @autoreleasepool сам, иначе автоосвобождённые объекты текут.**

Задача 17. Исключение через границу @autoreleasepool и в фоновом потоке

(Концептуальная, без файла — разбираем словами.)

Условие. Две ситуации.

1. Объект автоосвобождён **внутри** @autoreleasepool { ... }, и прямо оттуда бросают NSException, который ловят **снаружи** пула. Что станет с этим объектом и можно ли вернуть его наружу?
2. На фоновой GCD-очереди бросают NSException и **нигде** не ловят. Что будет с программой — упадёт только поток или весь процесс?

Подумай сам. Что происходит с пулом, когда исключение «разматывает» стек мимо его закрывающей скобки? И есть ли у фонового потока «верхний» ловец исключений, как у главного run loop?

Разбор, часть 1 — исключение через границу пула. Когда исключение летит вверх и стек **разматывается** (unwinding) мимо закрывающей скобки @autoreleasepool, пул на этой границе **сливается** — как при обычном выходе из блока. Все объекты, что держал только этот пул,

получают отложенный `release`. Если такой объект больше никто не удерживал сильно — он уничтожается прямо во время разматывания.

Практический вывод: **не рассчитывай вынести автоосвобождённый объект наружу из `@autoreleasepool` через исключение** — к моменту, когда ты поймаешь исключение снаружи, объект может быть уже мёртв (пул его слил при разматывании). Если объект нужен и за пределами пула — удержи его сильно **до** того, как пул сольётся (присвой сильной переменной снаружи или сделай `retain/copy`).

Разбор, часть 2 — исключение в фоне. В Objective-C `NSException` — это сигнал о **программерской ошибке** (вышел за границу массива, послал нераспознанный селектор), а **не** механизм управления потоком. Восстановимые ошибки передают через `NSError`, а не бросают исключениями.

Что до фонового потока: **никакого «верхнего» ловца у него нет**. Главный `run loop` в `AppKit/UIKit` на верхнем уровне как-то логирует пойманное, но обычный фоновый поток (и блок `GCD`) такой защиты не имеет. Непойманное `NSException` доходит до самого верха стека потока, `runtime` зовёт обработчик непоzymanных исключений и затем `abort()` — **падает весь процесс**, а не «только поток». Поток в общей памяти процесса; уронив его исключением, ты роняешь всё приложение.

Отсюда правила:

- `@try/@catch` — **не** инструмент повседневной обработки ошибок и тем более не «глушилка» багов. Поймать `NSException`, чтобы продолжить как ни в чём не бывало, опасно: после программмерской ошибки состояние программы уже могло поехать (полусделанная мутация, повисший `lock`).
- Исключение, пересекающее `@autoreleasepool`, сливает пул при разматывании — не жди живых автоосвобождённых объектов на той стороне.
- В фоновом потоке/блоке `GCD` непоzymanное исключение валит **весь процесс**. Для ожидаемых сбоев возвращай `NSError`, а исключения оставь на невосстановимые ошибки программиста.

В `code/` это не кладём: пример с непоzymanным исключением сознательно завершает процесс и сломал бы проверку.

Задача 18. Tagged pointers: почему == для маленьких NSNumber иногда прав

Условие. Два одинаковых **маленьких** NSNumber, созданных в рантайме, сравнивают через == (указатели). Потом — два одинаковых **больших**. Где == даст «равны», а где «нет», и почему?

```
NSNumber *a = [NSNumber numberWithInt:5];
NSNumber *b = [NSNumber numberWithInt:5];           // a == b ?
NSNumber *c = [NSNumber numberWithInt:1234567890123456789LL];
NSNumber *d = [NSNumber numberWithInt:1234567890123456789LL]; // c == d ?
```

Подумай сам. Всегда ли NSNumber — это объект в куче со своим адресом?

Разбор. На 64-битных системах маленькие NSNumber runtime хранит **прямо внутри указателя** — это **tagged pointer** («помеченный указатель»): само число закодировано в биты «адреса», отдельный объект в куче **не создаётся**. Два одинаковых маленьких числа получают **один и тот же** tagged-указатель — поэтому == для них случайно оказывается прав (да).

Большое число (1234567890123456789) в биты указателя не влезает — runtime создаёт **настоящий объект в куче**. Два таких равны по значению, но это **два разных** объекта по разным адресам, и == честно даёт нет.

Любопытная деталь: [a class] для tagged-числа всё равно показывает __NSCFNumber — снаружи это полноценный NSNumber, упаковка в указатель скрыта. И адреса tagged-указателей **обфусцированы** (runtime подмешивает случайную соль на каждый запуск), поэтому в выводе они выглядят как огромные «случайные» числа и меняются от запуска к запуску, в отличие от обычных кучных адресов.

Решение — code/senior-12-tagged.m:

```
маленькое 5: a == b (указатели)? да    класс=__NSCFNumber
           a=0x5012cb0de9e1ded7 b=0x5012cb0de9e1ded7 (адрес обфусцирован, меняется при запуске)
большое число: c == d (указатели)? нет  класс=__NSCFNumber
           c=0x6000007556c0 d=0x6000007556e0 (обычные адреса в куче)
isEqual: всегда верно: маленькие равны, большие равны
```

Смотри: маленькие 5 дали одинаковый указатель (== → да), большие — разные кучные адреса (== → нет). А isEqual: верен в **обоих** случаях.

Вывод, который и проверяют: == для NSNumber (и для объектов вообще) — **лотерея**, зависит от значения, разрядности и сборки. Сравнивай значения **только** через isEqual: / isEqualToNumber:. Tagged pointers — причина, по которой == иногда «случайно работает» и тем коварнее: на маленьких числах в тестах прошло, на больших в проде сломалось.

Отличие от Си. В Си `int` — это всегда само число в памяти, и `==` сравнивает значения. Здесь `NSNumber` — объект, `==` сравнивает указатели, и `runtime` ради экономии прячет маленькие числа в сам указатель. Поэтому «сравнение чисел» через `==` ведёт себя то как сравнение значений, то как сравнение адресов — и доверять ему нельзя.

Задача 19. Найди и почини утечку: цикл удержания через NSTimer

Условие. Объект `Ticker` заводит повторяющийся таймер на себя через `scheduledTimerWithTimeInterval:target:selector:...`. Объект выходит из области видимости, но `dealloc` не зовётся. Где цикл удержания и как починить?

[illegible]

Подумай сам. Как NSTimer относится к своему target — слабо или сильно? И кто держит сам таймер?

Разбор. `scheduledTimer...target:` **сильно удерживает** `target`, то есть `self`. А `self` сильно держит таймер в свойстве `timer`. И вдобавок запланированный таймер держит **run loop**. Кольцо:

```
run loop → timer —strong→ self —strong→ timer
```

Пока таймер не остановлен (`invalidate`), его держит `run loop`, таймер держит `self`, `self` держит таймер. Внешние ссылки на `Ticker` исчезли, а счётчик не падает до нуля — `dealloc` не сработает **никогда**. Коварство в том, что `weak` на свойстве `timer` тут не помог бы: сильную ссылку держит не оно, а сам таймер на `target`.

Чинить надо **связь таймер → self**. В современном API берут **блочный** таймер и захватывают self **слабо**:

[illegible]

```

    Ticker *strong = weakSelf;
    if (!strong) { [t invalidate]; return; }    // self ушёл – гасим таймер
    [strong tick:t];
  }];
}

```

Теперь таймер держит блок, а блок ссылается на self **слабо**. Когда внешний код отпускает Ticker, weakSelf становится nil, блок при очередном тике это видит и сам гасит осиротевший таймер (invalidate) — кольцо разорвано, dealloc срабатывает.

(Важно помнить: даже у блочного таймера self не освободится, пока таймер **активен** и держит блок, а run loop держит таймер. Поэтому таймер всё равно нужно invalidate — слабый захват лишь гарантирует, что само наличие таймера не образует **цикла** через self.)

Решение — code/senior-14-timer-cycle.m:

```

--- ПЛОХО: target/selector-таймер ---
  (dealloc для «bad» НЕ вызван – утечка через таймер)
--- ХОРОШО: блочный таймер + weak ---
  dealloc: Ticker «good» освобождён
  (dealloc для «good» вызван выше – цикл разорван)

```

У «bad» строки dealloc нет — объект застрял в кольце с таймером. У «good» dealloc напечатался: слабый захват self в блоке разорвал цикл, и осиротевший таймер сам погасился при следующем тике. По пропавшему dealloc, как обычно, утечку и ловят.

Отличие от Си. В Си «таймер» был бы структурой с указателем на callback-функцию — никаких счётчиков ссылок, никаких циклов удержания, только риск забыть free или позвать функцию по всяческому указателю. Здесь память считает ARC, и появляется новый класс ошибок: объект и таймер, сильно держащие друг друга. Лечится тем же weak, что и циклы с блоками и делегатами.

Что повторить перед Senior-собеседованием

Если на любой пункт отвечаешь не из модели, а «по памяти» — вернись к задаче.

- **isa и метаклассы.** Объект → класс → метакласс → корневой метакласс (замкнут на себя). —методы живут в классе, +методы — в метаклассе (задача 1).
- **Отправка сообщения.** [obj foo:5] = objc_msgSend(obj, sel, 5); SEL — имя, IMP — код, Method — их пара; поиск от isa вверх по superclass, потом кэш (задача 2).

- **Три шанса пересылки.** `resolveInstanceMethod:` (доращиваем метод) → `forwardingTargetForSelector:` (один другой получатель, дёшево) → `methodSignatureForSelector:` → `forwardInvocation:` (`NSInvocation`, много получателей, дорого) → `doesNotRecognizeSelector:` и падение (задачи 3, 4, 5).
- **Два корня.** `NSObject` (почти всё) и `NSProxy` (тонкий, для прокси — ничего лишнего не перехватывает) (задача 6).
- **Swizzling.** Только в `+load`, `dispatch_once`, сначала `class_addMethod` (не ломать родителя); глобален и опасен; со временем дерётся с KVO (задачи 7, 8).
- **Связанные объекты.** Категория добавляет методы, но не `ivar` — данные цепляем через `objc_setAssociatedObject` по адресу-ключу (задача 9).
- **KVO изнутри.** Подмена `isa` на `NSKVONotifying_`..., перехват сеттера, `-class` врёт; обязателен `removeObserver:` (задача 10).
- **Категории-дубли.** Два метода с одним именем — поведение не определено; категория добавляет, а не подменяет; префиксы (задача 11).
- **Потоки.** `atomic` ≠ потокобезопасная коллекция и ≠ атомарный `read-modify-write`; синглтон — `dispatch_once`; кэш — `concurrent`-очередь + `dispatch_barrier`; `dispatch_sync` на текущую `serial`-очередь — дедлок (задачи 12, 13, 14, 15).
- **weak и пулы.** `weak` потокобезопасен, но читай один раз в сильную локальную; у своего фонового потока нет пула — оборачивай в `@autoreleasepool` (задача 16).
- **Исключения.** Программерская ошибка, не управление потоком; пересекая пул, сливают его; непойманное в фоне валит весь процесс (задача 17).
- **Tagged pointers.** Маленькие `NSNumber` живут в самом указателе — `==` «случайно» прав; сравнивай через `isEqual:` (задача 18).
- **NSTimer.** Держит `target` сильно — цикл через `run loop`; лечится блочным таймером с `weak-Self` и `invalidate` (задача 19).

Документация Apple

- **Objective-C Runtime Programming Guide** — [developer.apple.com: Messaging\(objc_msgSend\),](https://developer.apple.com/Messaging(objc_msgSend),) [Dynamic Method Resolution\(resolveInstanceMethod:\),](https://developer.apple.com/DynamicMethodResolution(resolveInstanceMethod:),) [Message Forwarding\(forwardingTargetForSelector forwardInvocation:\).](https://developer.apple.com/MessageForwarding(forwardingTargetForSelector forwardInvocation:).)
- **Objective-C Runtime (API reference)** — developer.apple.com/documentation/objectivec (функции `object_getClass`, `class_getName`, `class_isMetaClass`, `class_getInstanceMethod`, `method_getImplementation`, `class_addMethod`, `method_exchangeImplementations`, `objc_setAssociatedObject`)
- `NSObject` — developer.apple.com/documentation/objectivec/nsobject (`class`, `+load`, `+resolveInstanceMethod:`, `-forwardingTargetForSelector:`, `-methodSignatureForSelector:`, `-forwardInvocation:`, `-doesNotRecognizeSelector:`).

- **NSProxy** — developer.apple.com/documentation/foundation/nsproxy (второй корневой класс для заместителей).
- **NSInvocation** — developer.apple.com/documentation/foundation/nsinvocation (упакованное сообщение для пересылки многим).
- **Key-Value Observing Programming Guide** — developer.apple.com (подписка, словарь change, что KVO делает с isa).
- **Concurrency Programming Guide** и **Dispatch (GCD)** — developer.apple.com/documentation/dispatch (dispatch_once, dispatch_sync/dispatch_async, dispatch_barrier_async, последовательные и параллельные очереди).
- **Advanced Memory Management Programming Guide** и **Transitioning to ARC Release Notes** — developer.apple.com (weak/strong, пулы автоосвобождения, циклы удержания, таймеры).
- **Exception Programming Topics** — developer.apple.com (NSException для программмерских ошибок, NSError для восстановимых).